# BIGQUERY SCENARIO BASED Q&A

BY - SHUBHAM WADEKAR

**Copyright © Shubham Wadekar. All rights reserved.**

**This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.**

**For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]**

**Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.**

**No job outcome is promised.**

© Shubham Wadekar

**1. Your team wants to migrate a 100TB Oracle data warehouse to BigQuery. How would you explain the benefits of BigQuery's architecture for this migration?**

Step 1. BigQuery is serverless and fully managed, so no infrastructure provisioning or tuning is required during migration of 100TB.

Step 2. Storage and compute are separated, allowing you to load 100TB cheaply into storage and scale compute independently for validation and queries.

Step 3. Columnar storage with high compression reduces the scanned data size, making queries on 100TB faster and cheaper.

Step 4. BigQuery's massively parallel execution engine processes queries across thousands of workers, so large validation queries finish quickly.

Step 5. Multiple ingestion options (bulk load from GCS, Data Transfer Service, Storage Write API) simplify moving 100TB reliably and efficiently.

Step 6. Partitioning and clustering allow you to organize large tables so queries scan only relevant data, reducing cost and improving performance.

Step 7. Time travel and snapshots provide rollback safety during migration, helping you recover quickly if issues occur.

Step 8. Built-in security, IAM, and compliance support ensure governance over sensitive enterprise data as it moves from Oracle.

Step 9. Cost model supports both on-demand queries and flat-rate slots, giving flexibility to manage expenses for large-scale workloads.

Step 10. Standard SQL compatibility lets teams reuse queries with minimal changes, accelerating adoption after migration.

**2. A stakeholder asks why BigQuery doesn't need indexes like traditional RDBMS. How do you explain this?**

Step 1. In traditional RDBMS, data is stored row-by-row, so without indexes the system must scan every row to find matching records. Indexes are created manually to speed up lookups but add maintenance overhead.

Step 2. BigQuery uses columnar storage, meaning data is stored by columns instead of rows. Queries that only need a few columns read only those columns, reducing scanned data dramatically without needing indexes.

Step 3. BigQuery's execution engine automatically parallelizes queries across thousands of workers. Even if the system scans large volumes of data, it does so in parallel, making "full scans" much faster compared to traditional databases.

Step 4. Partitioning in BigQuery lets you split large tables by date or another key, so queries automatically skip partitions that aren't relevant. This works like automatic pruning, avoiding the need for traditional indexes.

Step 5. Clustering in BigQuery organizes data within each partition based on selected columns. Queries filter faster because relevant blocks of data are colocated, again achieving the benefits of indexing without separate index structures.

Step 6. Because of this combination - -columnar storage, parallel processing, partitioning, and clustering - -BigQuery eliminates the need for explicit indexes and still delivers high performance on very large datasets.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**3. Your query performance drops as data grows to billions of rows. How does BigQuery's architecture still handle this scale?**

Step 1. BigQuery stores data in a columnar format where values of the same column are stored together. This allows queries to read only the needed columns instead of entire rows, which greatly reduces the amount of data scanned.

Step 2. Tables are automatically divided into many small blocks stored in distributed storage. These blocks can be read in parallel by multiple workers, which enables BigQuery to scan billions of rows quickly.

Step 3. Query execution uses a tree structure called the Dremel execution engine. Leaf nodes scan and filter data, intermediate nodes combine partial results, and the root node produces the final output. This distributed aggregation prevents the need to move all raw data across the network.

Step 4. Storage and compute are separated. Storage can grow to petabytes without affecting query performance, while compute slots scale dynamically depending on the query size.

Step 5. BigQuery applies optimizations such as predicate pushdown (filters applied at read time), column pruning (unused columns skipped), and vectorized execution (processing multiple values in a single CPU operation).

Step 6. Partitioning and clustering further limit the amount of data scanned. Partitioning allows queries to read only specific partitions such as dates, while clustering keeps related rows close together to speed up filtering and aggregations.

Step 7. Query slots are allocated dynamically so that larger queries automatically get more compute resources. This elasticity ensures consistent performance even as data size grows.

Step 8. Repeated queries benefit from caching and materialized views, which reduce the need to recompute results on large tables.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**4. You are asked to compare Hadoop-based Hive queries vs BigQuery. How do you highlight BigQuery's advantages?**

Step 1. Hive queries run on clusters that must be provisioned, scaled, and maintained, while BigQuery is serverless and fully managed with no infrastructure management.

Step 2. Hive typically stores data in HDFS and depends on file formats such as ORC or Parquet for columnar efficiency, while BigQuery natively uses optimized columnar storage with compression and pruning built in.

Step 3. Hive query execution is based on MapReduce or Tez, which introduce job startup delays and higher latency. BigQuery uses a Dremel-based execution engine that parallelizes queries across thousands of nodes with much lower overhead.

Step 4. Scaling Hive requires adding or resizing cluster nodes and careful resource management. BigQuery automatically allocates and scales compute slots for queries, eliminating manual scaling.

Step 5. Hive requires managing Hadoop ecosystem components such as YARN, HDFS, and cluster monitoring. BigQuery removes this operational burden because Google manages hardware, scaling, replication, and availability.

Step 6. Hive is largely batch-oriented, while BigQuery supports both batch and real-time use cases, including streaming ingestion directly into tables.

Step 7. BigQuery provides advanced features like materialized views, federated queries, GIS functions, and built-in machine learning, whereas Hive usually requires integrating external tools for these capabilities.

Step 8. Hive costs are tied to running and maintaining clusters, while BigQuery offers on-demand pricing by bytes processed or flat-rate reservations, which can lower total cost of ownership for analytics workloads.

**Question 5. Your company has both batch and real-time needs. How does BigQuery's architecture support both in the same platform?**

Step 1. High-level summary - BigQuery supports both batch and real-time analytics by separating storage from compute and providing a distributed, columnar storage engine plus a massively parallel query engine; this lets you ingest data quickly (real-time) and also run large, cost-efficient batch queries over the same datasets.

Step 2. Storage layer explained - Data in BigQuery is stored in a columnar format which means only the columns needed by a query are read - this makes analytic scans fast and cost-efficient for batch queries. Tables can be partitioned (usually by ingestion time or date) and clustered (grouping similar rows) to further reduce the amount of data scanned - very important for batch cost control. The storage is durable and distributed - so data written by streaming or batch loads becomes available to the query engine.

Step 3. Compute / query engine explained - BigQuery's query engine (a Dremel-style, massively parallel system) dynamically allocates compute across many workers to scan, aggregate, and join data. Because compute is decoupled from storage you can elastically run large batch queries without impacting ingestion, and small interactive queries can run concurrently against the same data.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Step 4. How streaming (real-time) ingestion works - BigQuery supports low-latency ingestion via streaming inserts (for example tabledata.insertAll or pipelines from Pub/Sub → Dataflow → BigQuery). Streaming rows land in a streaming buffer that is queryable within seconds - enabling near-real-time dashboards and alerts. Streaming is designed for speed and immediacy - but typically has a higher per-row cost than bulk loads.

Step 5. How batch ingestion works - For cost-efficient large-volume loads you use batch load jobs (for example load from files in Cloud Storage or scheduled Dataflow jobs) which write directly to table storage and can populate partitions. Batch loads are cheaper per-row and ideal for historical backfills and daily aggregations - use partitioned tables to limit scanned data and lower cost.

Step 6. Unified querying and consistency - Queries can transparently read both streaming-buffered rows and persisted partitions in the same SQL statement, so users don't need separate systems for real-time vs batch analytics. Use partitioned tables, partition pruning, and clustering so queries touch only necessary data. Be aware streaming-buffered rows may have temporary limitations (for very recent data operations like metadata changes) - but they remain queryable for analytics.

Step 7. Operational & cost best practices - Balance trade-offs by using streaming only for low-latency needs and micro-batching for high-volume near-real-time use to reduce cost - e.g. accumulate a few seconds/minutes of events before writing. Use partitioned + clustered tables to reduce scan costs, consider separate ingestion tables or staging partitions during heavy loads, and use slot reservations or flat-rate pricing if you need predictable query performance. Monitor ingestion lag, slot utilization, and query bytes scanned with Cloud Monitoring and BigQuery audit logs to tune the pipeline

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 6. You have a 5TB log table with daily inserts. How would you design the table with partitioning to reduce costs?**

Step 1. Choose the partition key and type - Partition the table by date (use a DATE or TIMESTAMP field representing the event date, e.g., event_date or event_timestamp) so each day's rows go into a single partition. If your logs don't have a reliable event timestamp, use ingestion-time partitioning (_PARTITIONTIME) as a fallback. Daily partitions match your daily insert pattern and make pruning straightforward.

Step 2. Use ingestion pattern to pick column-partitioning vs ingestion-time - If the log producer reliably sets event timestamps, use column-based DATE partitioning (better control and correctness). If timestamps are missing or unreliable and you need fastest setup, use ingestion-time partitioning. Column-based partitioning is preferable because it preserves event semantics and supports backfills more cleanly.

Step 3. Add clustering on high-cardinality, commonly-filtered columns - After partitioning by day, cluster the table on 2–3 frequent filter/join columns (for example service_name, status_code, user_id, or request_path depending on your query patterns). Clustering reduces the number of bytes scanned within a partition for selective queries and speeds aggregations.

Step 4. Set partition expiration and lifecycle rules - Apply a sensible partition expiration (for example 30/90/365 days depending on retention needs) to automatically drop older partitions and reduce storage cost. If you must keep long-term raw logs, export older partitions to cheaper storage (GCS) and delete from BigQuery.

Step 5. Choose ingestion method to control cost - For daily bulk inserts prefer batched load jobs (GCS → BigQuery) or micro-batches (small time windows) rather than per-row streaming, because streaming has higher per-row costs and can inflate the streaming buffer. If you need near-real-time, use short micro-batches (e.g., 1–5 minute files) to balance latency and cost.

Step 6. Design partition sizes and avoid tiny/huge partitions - Aim for partition sizes that avoid many tiny partitions and also avoid single huge partitions. With 5TB total and daily inserts, estimate average daily size and ensure it's reasonable (e.g., tens of GB/day is fine). If one day can be huge, consider sub-partitioning strategies like DATE + hourly sharding in an ingestion staging table, then compact into daily partitions.

Step 7. Enforce query best-practices and protect against full-table scans - Require that dashboards and analysts include a partition filter on event_date in their queries; teach them to use _PARTITIONTIME or WHERE event_date BETWEEN .... Use query cost controls (labels, quotas, cost alerts, or authorized views) to prevent accidental full-table scans.

Step 8. Monitor and iterate - Use BigQuery's INFORMATION_SCHEMA and Cloud Monitoring to track partition sizes, bytes scanned per query, streaming buffer usage, and slot utilization. If certain partitions are hotspots or queries still scan too much data, refine clustering keys, add materialized views for common aggregates, or introduce summary tables to avoid scanning raw partitions repeatedly.

Step 9. Practical extras for operational reliability - Use partition decorators for point-in-time reads and backfills (e.g., tablename$YYYYMMDD) and maintain a small staging table for heavy backfill jobs to avoid locking or impacting production partitions. Document partitioning and query patterns for your team so everyone writes partition-pruned queries.

Step 10. Cost/benefit trade-offs summary - Partitioning by day + clustering greatly reduces bytes scanned and lowers query cost, partition expiration cuts storage cost, and batched ingestion lowers ingest cost; accept that streaming and very wide query patterns still incur costs, so combine partitioning with clustering, summaries, and access controls to achieve predictable, low-cost analytics.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**Question 7. A dashboard query always filters on region and date. How would you use clustering and partitioning together?**

Step 1. Pick the partition key - Partition the table by date (use a DATE or TIMESTAMP column such as event_date) because the dashboard filters by date daily; partitioning by date enables partition pruning so queries scan only the relevant day(s).

Step 2. Choose clustering columns and order - Cluster the table on region first (since the dashboard always filters on region), then add 1–2 other high-cardinality or frequently-filtered columns (for example service_name or user_id) so that within each date partition rows with the same region are colocated and scans are minimized.

Step 3. How partitioning + clustering interact - Explain that partitioning reduces the number of partitions scanned (date-level pruning) while clustering reduces the bytes scanned inside each selected partition by physically grouping similar region values together; together they drastically cut bytes scanned for queries that filter on both date and region.

Step 4. Design queries to enable pruning - Ensure dashboard queries include a filter on the partition column (e.g., WHERE event_date = '2025-09-12' or BETWEEN ... ) and a direct filter on region (e.g., WHERE region = 'ap-south1'); avoid wrapping the partition column in functions or expressions that prevent pruning.

Step 5. Evaluate clustering cardinality and maintenance - Choose clustering keys with moderate-to-high cardinality so clustering is effective; understand clustering maintenance is automatic but can become less optimal with heavy streaming inserts, so periodically run table reorganizations (e.g., export/compact into new partitioned+clustered table) if clustering quality degrades.

Step 6. Use table design patterns for dashboards - For extremely hot partitions or very high query concurrency, consider a small aggregated summary table (daily by region) or materialized views that pre-aggregate metrics per date+region to make dashboard queries cheaper and faster.

Step 7. Enforce best practices and protect costs - Create query templates or authorized views that always include the date+region filter, set cost controls and monitored alerts for queries scanning large amounts of data, and document the required filters for dashboard authors.

Step 8. Monitor and iterate - Use INFORMATION_SCHEMA, BigQuery job statistics, and Cloud Monitoring to track bytes scanned per dashboard query, partition sizes, and clustering effectiveness; adjust clustering keys, partition granularity, or add summary tables if metrics show suboptimal scan patterns.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 8. You notice queries are scanning full partitions. What steps would you take to fix this?**

Step 1. Identify offending queries and inspect query plans - Use BigQuery JOBS and INFORMATION_SCHEMA to list expensive queries and examine their query plans and referenced tables to confirm why partitions are fully scanned (e.g., missing partition filter, functions on partition column, or wrong column used).

Step 2. Verify partition filters are present and correct - Check that queries include a direct predicate on the partition column (e.g., WHERE event_date = 'YYYY-MM-DD' or _PARTITIONTIME BETWEEN ...); if not, modify queries to include explicit partition filters.

Step 3. Fix predicate anti-patterns that prevent pruning - Replace expressions that wrap the partition column (for example DATE(event_timestamp) = ...) with equivalent predicates that allow pruning (for example event_timestamp BETWEEN '2025-09-12T00:00:00' AND '2025-09-12T23:59:59') or use partition decorators (table$YYYYMMDD).

Step 4. Check for column type mismatches and implicit casts - Ensure the partition column type in the query matches the table (DATE vs TIMESTAMP); implicit casts can disable pruning, so cast explicitly to the partition column type on the literal side instead of on the column.

Step 5. Add clustering or refine clustering keys - If queries filter on other columns but still scan many rows within a partition, add clustering on those filter columns (e.g., region, status_code) to reduce bytes scanned inside partitions.

Step 6. Introduce summary tables or materialized views - For repeated heavy aggregations that still read entire partitions, create daily summary tables or materialized views (pre-aggregated by partition keys) so dashboards read small precomputed results instead of raw partitions.

Step 7. Implement query guards and educate users - Enforce sensible defaults (require partition filters in dashboards), create authorized views that encapsulate proper filters, set query cost quotas/alerts, and teach analysts how to write partition-pruned queries.

Step 8. Reorganize data if necessary - If partitions are extremely large or poorly clustered, perform a controlled rewrite: export and reload data into correctly partitioned+clustered partitions or run a compaction job to improve data locality and clustering effectiveness.

Step 9. Monitor after changes - After applying fixes, monitor bytes scanned per query, partition scan counts, and query latencies to verify improvements; iterate on partition granularity, clustering keys, or additional summarization if problems persist.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 9. Your partitioned table is hitting the maximum 4000 partitions. What strategies can you apply?**

Step 1. Diagnose the partition usage - Check how partitions are created (daily, hourly, per-event), count partitions by querying INFORMATION_SCHEMA.PARTITIONS, and identify whether many tiny partitions are empty/short-lived or whether partitioning granularity is unnecessarily fine for your query patterns.

Step 2. Coarsen partition granularity - If you currently use daily partitions, move to monthly partitions (or if daily but low retention, use weekly) so the same time range uses far fewer partitions. Create a new column like partition_month = DATE_TRUNC(event_date, MONTH) and partition on that field, then backfill recent data.

Step 3. Use logical table sharding by time ranges - Split the data across multiple tables (for example logs_2024, logs_2025) where each table uses its own partitions. Query across them with a UNION ALL view or a partitioned view so consumers can query transparently while each table stays under the 4000 limit.

Step 4. Employ partitioned views or date-based routing - Create a view that unions several partitioned tables and use query-time logic to only touch the relevant tables/partitions - this avoids a single table exceeding partition limits while keeping a unified query surface.

Step 5. Consider integer-range partitioning or custom bucketing - If time isn't the only partition dimension and you can map keys into ranges, use integer-range partitioning (e.g., a bucket_id) to reduce partition count while preserving pruning for common filters.

Step 6. Use clustering to reduce reliance on many tiny partitions - If partitions were created to speed selective queries on other dimensions (not time), move to fewer partitions and add clustering on those selective columns (region, service, status) so queries still scan less data within a partition.

Step 7. Implement retention and automatic cleanup - Set partition expiration policies to drop old partitions automatically so you don't accumulate to 4000; for long-term retention export old partitions to GCS and delete them from BigQuery to free partition slots.

Step 8. Use table rotation for hot data - Keep a small hot table for current high-ingest periods and periodically merge/compact into the historical partitioned tables; this helps avoid creating many ephemeral partitions from streaming inserts.

Step 9. Plan migration and backward-compatibility - Create a migration plan: create new coarser-partitioned tables, backfill recent data, update ETL and query layers, and add authorized views to avoid breaking dashboards. Test query performance and costs before deprecating the old table.

Step 10. Monitor and iterate - After changes, monitor partition counts, query bytes scanned, and cost. If issues persist, combine strategies above - e.g., monthly partitions + clustering + yearly table rotation - until partition limits and performance goals are met.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 10. A dataset's query performance is degrading because partitions are too large. How would you redesign it?**

Step 1. Measure the problem and query patterns - Use INFORMATION_SCHEMA, job statistics, and query logs to find which queries scan large partitions, which partitions are oversized, and which filters users commonly apply (time ranges, region, service, user_id).

Step 2. Reduce partition size by increasing partitioning granularity - If partitions are daily and too large, move to smaller-granularity partitions such as hourly or sub-day buckets for high-ingest sources. Create a new partition key (for example event_hour = TIMESTAMP_TRUNC(event_timestamp, HOUR)) and migrate recent hot data into hour-partitioned tables.

Step 3. Add clustering on frequently-filtered columns - After partitioning by time, cluster on columns users filter by (for example region, service_name, status_code). Clustering groups similar values inside partitions and reduces bytes scanned for selective queries.

Step 4. Introduce summary tables and materialized views - For dashboards or recurring heavy aggregations, build daily/hourly aggregated tables or materialized views keyed by partition keys - queries then read small pre-aggregated data instead of scanning huge raw partitions.

Step 5. Use hybrid sharding or multi-table design for hotspots - If certain days generate huge spikes, isolate those into separate tables (shards) for that time range or use a hot-table that gets compacted later - this prevents single partitions from growing unbounded.

Step 6. Optimize ingestion to improve locality - Compact small files and avoid many tiny micro-partitions by batching writes (batch loads or micro-batching) rather than per-row streaming where feasible, so storage layout and clustering stay effective.

Step 7. Repartition/compact existing data - Perform controlled rewrite jobs: export and reload data into the new partitioning + clustering scheme or use CTAS (create table as select) to rebuild partitions with better data locality, keeping downtime minimal by working on recent partitions first.

Step 8. Enforce query and schema best practices - Require partition filters in queries, document the correct partition column and formats, avoid functions on the partition column that prevent pruning, and provide parameterized query templates or authorized views to ensure partition-pruned access.

Step 9. Leverage resource management and caching - Use BigQuery slot reservations if concurrency is an issue, enable result caching for repeated dashboard queries, and consider BI Engine for sub-second dashboard performance for supported query patterns.

Step 10. Monitor, test, and iterate - Continuously track partition sizes, bytes scanned, query latency, and user patterns. If new hotspots appear, iterate on partition granularity, clustering keys, or add further summarization so performance stays predictable and cost-efficient.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 11. A developer is running SELECT * queries on a 20TB table and the project costs are spiking. How do you fix this?**

Step 1. Identify which queries are causing the cost spike by checking BigQuery's INFORMATION_SCHEMA.JOBS views or Cloud Monitoring logs to see who is running SELECT * on the 20TB table, how often, and how many bytes are scanned.

Step 2. Apply immediate controls by setting maximum bytes billed per query, enabling cost quotas, or temporarily restricting the developer's permissions so the uncontrolled SELECT * queries stop consuming excessive resources.

Step 3. Provide controlled access through authorized views that expose only the necessary columns and apply default filters such as date ranges. This ensures queries automatically avoid full-table scans.

Step 4. Educate the developer and team to avoid SELECT * and instead query only needed columns. Share query templates and enforce coding standards that require explicit column selection and partition filters.

Step 5. Enforce best practices technically by enabling the "require partition filter" option on partitioned tables and setting up query cost alerts to prevent accidental full scans in the future.

Step 6. Redesign the schema usage by creating narrow tables or column-specific extracts for frequent analytics use cases so dashboards and analysts don't need to scan the full 20TB raw table.

Step 7. Optimize the raw table with partitioning by date and clustering on commonly filtered columns so queries scan only relevant slices of data.

Step 8. Build summary tables or materialized views that pre-aggregate common metrics, allowing analysts to query smaller datasets instead of raw logs.

Step 9. Update dashboards and applications to use the summary tables or views instead of querying the full raw table directly.

Step 10. Continuously monitor query costs and bytes scanned per user to ensure the issue does not repeat, and refine cost controls, summaries, and permissions as needed.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**Question 12. A reporting dashboard is slow due to repeated aggregations. How would you optimize it in BigQuery?**

Step 1. Analyze the dashboard's queries using INFORMATION_SCHEMA and query history to see which aggregations are repeated and how many bytes they scan each time.

Step 2. Create scheduled summary tables that precompute daily or hourly aggregates so the dashboard queries a much smaller dataset instead of recalculating metrics from raw data every refresh.

Step 3. Use materialized views for repeated aggregations with stable SQL patterns, since BigQuery maintains them incrementally and queries on them return much faster.

Step 4. Enable BI Engine to cache and accelerate query results in memory, giving sub-second performance for supported dashboard queries.

Step 5. Partition raw tables by time (such as event_date) and cluster by frequently filtered fields (such as region or product) so queries prune partitions and scan fewer bytes when accessing raw data.

Step 6. Create optimized reporting tables with only the required columns and, if necessary, denormalize common joins so the dashboard reads one compact dataset instead of running multiple joins and scans.

Step 7. Materialize heavy joins by precomputing and storing joined datasets instead of letting the dashboard execute expensive joins repeatedly.

Step 8. Reduce unnecessary refresh frequency in the dashboard and update summary tables incrementally, adding only recent data rather than recomputing the full dataset each time.

Step 9. Monitor dashboard performance metrics and query scan statistics to validate improvements and adjust partitioning, clustering, or BI Engine capacity as usage grows.

Step 10. Iterate continuously, refining the design of summary tables, materialized views, and caching strategy to keep performance and cost predictable at scale.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 13. Your team needs COUNT(DISTINCT) on billions of rows for daily reports. How would you balance accuracy vs performance?**

Step 1. Start by checking the business requirement for accuracy. If the finance or operations team needs exact numbers, you must use COUNT(DISTINCT) which is slower and costly. If they only need approximate counts within a small error range, you can choose faster options.

Step 2. For very large datasets, use BigQuery's APPROX_COUNT_DISTINCT(). This works much faster than the exact distinct and scans less data. It usually gives results within 1–2% error, which is good enough for many reporting dashboards.

Step 3. If the approximation needs to be more accurate, you can tune the precision. BigQuery internally uses HyperLogLog (HLL) sketches, which allow you to adjust the error margin by changing precision settings. Higher precision uses more storage but still performs faster than exact distinct.

Step 4. Combine both methods: for business-critical groups where accuracy is very important (like top customers or regulatory reports), run exact distinct; for less important groups, use approximate. This way you save cost and time while keeping critical metrics accurate.

Step 5. Instead of calculating distinct counts every day from scratch, store pre-built distinct summaries or HLL sketches during data ingestion. These can be merged quickly when running daily reports, which avoids rescanning billions of rows.

Step 6. Create summary tables or materialized views for distinct counts by date, region, or other common dimensions. Reports can then query these smaller pre-aggregated tables instead of the entire raw dataset.

Step 7. Regularly validate the approximate results by comparing them with exact distinct counts on a smaller dataset. This ensures the error margin is within the acceptable business range.

Step 8. Finally, communicate clearly to the reporting team which numbers are exact and which are approximate, so they understand the trade-off between speed and accuracy.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 14. Your finance team wants predictable monthly BQ costs. Would you recommend on-demand or flat-rate slots? Why?**

Step 1. First understand the difference: On-demand pricing charges you per terabyte of data scanned. The more queries you run, the higher the monthly bill. Flat-rate pricing charges a fixed monthly fee for a set number of slots (compute power).

Step 2. If finance wants a stable, predictable bill every month, flat-rate is the better choice because you always pay the same amount no matter how much data you scan.

Step 3. Flat-rate works best if your company runs a lot of queries regularly, such as daily dashboards, ETL jobs, or reporting pipelines. In this case, you get both predictable cost and dedicated performance.

Step 4. If your query usage is very low or unpredictable (sometimes very heavy, sometimes very light), on-demand might be cheaper because you only pay when you use BigQuery.

Step 5. Sometimes a hybrid is best: keep flat-rate slots for daily reporting and batch workloads, and allow on-demand for ad-hoc or one-time exploration queries. This gives predictability for finance and flexibility for developers.

Step 6. Flat-rate also helps performance because you can allocate slots to different teams, ensuring critical jobs always have resources. This prevents delays during high concurrency.

Step 7. Before deciding, analyze your last few months of query costs and slot usage. If you consistently spend more than or equal to flat-rate pricing, then flat-rate is the safer option.

Step 8. Start with flex slots (short-term commitments) to test flat-rate usage, measure slot utilization, and then move to monthly or yearly flat-rate once you're confident it matches your workload.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**Question 15. You see queries spilling to disk frequently. How do you optimize them?**

Step 1. First check BigQuery job execution details to confirm why queries are spilling. Spills happen when there isn't enough memory to handle operations like large joins, group-bys, sorts, or DISTINCT. This slows queries significantly.

Step 2. Look at the join logic. If the query is joining huge tables, try to filter data earlier in the query so only the needed rows are joined. Avoid cross joins or unnecessary large joins. This reduces the amount of data BigQuery must shuffle.

Step 3. Always select only the required columns. A SELECT * query brings unnecessary columns, which increases the data shuffled between workers and causes more memory usage. Adding proper filters also helps reduce shuffle size.

Step 4. Use partitioning and clustering on large tables. Partitioning ensures queries read only the relevant partitions (like a single day instead of the whole table). Clustering keeps similar rows close together, so less data is scanned and shuffled inside partitions.

Step 5. Break large queries into smaller steps. For example, write results of an intermediate stage into a temporary table, then run the next part of the query on this smaller dataset. This avoids one single query from doing all heavy work at once.

Step 6. For aggregations, consider using approximate functions (like APPROX_TOP_COUNT, APPROX_QUANTILES) if exact results are not required. They use less memory and reduce spills.

Step 7. If the issue is resource limits, check your slot allocation. Queries may be spilling because they don't have enough slots/memory. Reserving more slots for heavy workloads can help.

Step 8. Rewrite costly operations like ORDER BY and DISTINCT. For example, if you only need the top N rows, use ORDER BY ... LIMIT N instead of sorting the whole dataset. For deduplication, use ROW_NUMBER over partitions instead of SELECT DISTINCT on billions of rows.

Step 9. After applying these fixes, monitor queries again. Use job statistics to check shuffle size, memory usage, and if spills are reduced. Keep tuning queries and table design until spills are rare and performance is stable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 16. You are designing a schema for customer orders with nested items. Would you use normalized or denormalized schema? Why?**

Step 1. Start with the use-case - -ask whether the workload is analytical (reporting, aggregations, few updates) or transactional (many single-row updates, strict normalization). BigQuery is built for analytics, so that matters for the choice.

Step 2. Prefer denormalized with nested items for analytics - -model orders as one row per order and embed items as a repeated RECORD/ARRAY field (items: ARRAY<STRUCT<item_id, qty, price, ...>>). This keeps related data together and avoids joins.

Step 3. Explain query simplicity - -with nested items you can write queries that UNNEST only when needed, or aggregate across items without costly joins. That reduces bytes scanned and often runs much faster in BigQuery.

Step 4. List performance and cost benefits - -fewer joins → less shuffle and fewer scanned bytes → lower cost and faster queries. Columnar storage reads only needed columns, so nested fields are efficient for analytic access patterns.

Step 5. Note drawbacks and update complexity - -heavy per-item transactional updates (update a single item frequently) are harder with denormalized records because you rewrite the row. If your application needs many item-updates, normalization or a hybrid approach might be better.

Step 6. Offer hybrid patterns - -keep a denormalized order table for analytics and a normalized event/transaction table for operational writes. Use ETL to sync/merge into the denormalized analytic table on a schedule.

Step 7. Show practical modeling tips - -use appropriate types (STRUCT/RECORD, REPEATED), choose clear field names, add summarized columns (order_total, item_count) to avoid flattening unnecessarily, and add ingestion-time metadata (ingest_time, source) for traceability.

Step 8. Consider maintenance and governance - -document how to query (UNNEST patterns), provide views for common queries, and plan compaction/backfill jobs when schema changes are needed so consumers aren't broken.

Step 9. Final rule of thumb - -for BigQuery analytics prefer denormalized nested schema for orders+items for simplicity, cost, and speed; choose normalization only when your workload has frequent per-item transactions or strict transactional integrity needs.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**Question 17. A new column needs to be added to an existing production table. How would you handle it without breaking queries?**

Step 1. Plan the change and communicate - -announce the new column to teams, explain purpose and timing, and list any consumers that must change. Communication avoids surprise breakage.

Step 2. Add the column as nullable (or with default semantics in your ETL) - -in BigQuery add a new nullable column so existing rows don't need the new value and existing queries keep working. Do not add a REQUIRED column because that can break writes.

Step 3. Use a phased rollout - -first add the column and keep ETL writing nulls; then update downstream ETL or producers to start populating the new column; finally backfill historical data if needed. Each phase is separate and reversible.

Step 4. Provide safe access via views - -create or update an authorized view that projects only stable columns (or exposes a computed column) so existing users continue querying the view and only opt-in consumers use the new column. This decouples schema change from clients.

Step 5. Backfill carefully - -if historical values are needed, create a backfill job that writes to a new table or uses a controlled CTAS (create table as select) or write-time update; avoid heavy simultaneous backfills on production partitions - -do it incrementally.

Step 6. Test against staging - -validate queries, dashboards, and ETL in a staging environment with the new column populated to catch issues (type mismatches, unexpected nulls) before production rollout.

Step 7. Avoid renaming or dropping existing columns at the same time - -renames/drops are breaking changes; do them in separate steps and coordinate consumers. If you must rename, add the new column, update consumers, then remove the old column later.

Step 8. Update documentation and clients - -publish the updated schema, sample queries, and change logs; ask teams to add the new column only when they are ready. Monitor for any failing queries after rollout and revert or fix quickly if needed.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 18. A JSON feed keeps adding new fields each month. How would you design the schema for BigQuery ingestion?**

Step 1. Treat the feed as semi-structured - -accept that fields will evolve. Plan to store raw JSON plus a flattened/typed subset for common queries. This gives flexibility and query performance.

Step 2. Ingest raw JSON into a single column of type JSON (or STRING) - -store the full payload in a JSON column so you never lose unknown fields and can reprocess later if schema changes are discovered.

Step 3. Extract and store known fields as typed columns - -parse out stable, commonly-used fields into explicit columns (STRING, INT, TIMESTAMP) for fast querying, partitioning, and clustering. Keep these columns nullable to handle missing values.

Step 4. Use a flexible pattern for evolving fields - -for fields that come and go or are dynamic, either (a) keep them nested inside the JSON column, or (b) extract them into a repeated key-value table (key, value) for ad-hoc queries. Choose key-value when you need to query unknown keys often.

Step 5. Automate schema detection and evolution in your pipeline - -build ETL logic (Dataflow/Beam or custom job) that detects new top-level fields and either alerts engineers, auto-adds new nullable columns, or writes them into the JSON/key-value store. Keep humans in the loop for critical schema decisions.

Step 6. Flatten on demand for analytics - -create scheduled jobs or views that flatten JSON into typed columns for reporting when a field becomes stable; use materialized views or summary tables for fields used frequently by dashboards.

Step 7. Partition and cluster using stable keys - -partition on a stable field like event_date (derived from parsed timestamp) and cluster on high-selectivity keys so queries remain efficient even with varying schema.

Step 8. Archive raw feeds and enable replay - -keep original files in GCS (or the JSON column) so you can re-run parsing when new fields arrive or parsing logic improves. This avoids data loss when evolving schemas.

Step 9. Document schema evolution and versioning - -keep a registry of fields, their types, and when they first appeared; version your ingestion pipeline and track breaking changes so downstream consumers know when to adapt.

Step 10. Monitor schema drift and alert - -add automated checks that report new or missing fields monthly, so engineers can decide whether to promote fields to typed columns, keep them in JSON, or retire them.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**Question 19. A downstream team requires strict NOT NULL constraints, but upstream data is inconsistent. How do you design this?**

Step 1. Assess requirements and sources - Talk to the downstream team to understand which fields must be NOT NULL, why (SLAs, downstream joins, legal), and inspect upstream data to see how often and why those fields are missing or malformed.

Step 2. Add a validation layer at ingestion - Implement an ETL step (Dataflow, Cloud Functions, or a pre-load job) that validates incoming records and separates them into "clean" and "dirty" streams so you can treat each differently.

Step 3. Create a clean, NOT NULL production table - Load only validated records into the production table that enforces NOT NULL semantics by ensuring the ETL fills required fields or drops/redirects bad rows; keep the table schema with those columns non-nullable in your documentation and contracts.

Step 4. Store raw/dirty data separately - Write all original incoming records (including those with missing fields) to a raw or staging table or to GCS as JSON/Avro so you never lose data and can reprocess later if upstream fixes arrive.

Step 5. Implement defaulting and enrichment rules - Where business-appropriate, fill missing values with deterministic defaults, computed values, or lookups (for example map missing region via IP lookup) and clearly tag these rows with metadata (e.g., _is_defaulted = TRUE) so consumers know they were filled.

Step 6. Provide an Exceptions / Quarantine table - Redirect rejected or repairable rows into a quarantined table with error reasons and payload, and build monitoring/alerts so data owners can fix upstream sources or trigger reprocessing.

Step 7. Offer views for different consumers - Expose a strict view over the clean table that guarantees NOT NULL to downstream users, and another permissive view that unions cleaned + flagged rows for troubleshooting or audits. Use authorized views to control access

.

Step 8. Automate backfills and reprocessing - When upstream fixes arrive, reprocess raw/quarantine data to populate the clean table and backfill NOT NULL fields, keeping an idempotent pipeline to avoid duplicates.

Step 9. Monitor and close the loop - Create dashboards and alerts for missing-field rates, quarantine growth, and SLA breaches; share reports with upstream owners and agree on remediation timelines.

Step 10. Document contracts and evolve the schema - Publish a data contract stating required fields, types, and SLAs; when schema changes are needed, use versioning and staged rollouts so downstream teams remain protected.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 20. You are asked why BigQuery often recommends denormalized schemas. How do you justify this with an example?**

Step 1. State the core reason simply - BigQuery is optimized for analytic reads, not transactional joins; denormalized schemas reduce joins, lower data scanned, and speed queries, which saves money and improves latency.

Step 2. Give a concrete example setup - Imagine orders and order_items tables: normalized design has one row per order and many rows per item in a separate table; a denormalized design stores orders with an items array inside each order row.

Step 3. Show how a typical analytic query changes - A sales report that needs order totals by product in normalized schema must join orders and order_items and aggregate across both tables; with denormalized nested items you UNNEST the items inside orders and aggregate directly, avoiding a heavy join.

Step 4. Explain performance and cost benefits - Avoiding joins means less shuffling across workers and fewer bytes read, so queries run faster and cost less; nested fields are stored column-wise so you only read the columns you need.

Step 5. Mention simplicity for analysts - Denormalized tables let analysts write simpler SQL (fewer joins) and reduce chances of mistakes in join keys or duplicates when joining very large tables.

Step 6. Note common caveats - If the application needs frequent single-item updates or strict normalization for OLTP, denormalization can be harder to update; in those cases use a hybrid approach.

Step 7. Offer a hybrid pattern example - Keep operational normalized tables for writes, and run ETL to build a denormalized analytic table (orders with items array) for reporting; this gives best of both worlds.

Step 8. Conclude with a rule of thumb - For analytics and BI on BigQuery, prefer denormalized, nested schemas when it reduces joins and aligns with query patterns; use normalization only when transactional guarantees or frequent per-row updates require it.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 21. You need to load 5TB historical data from GCS daily. Which file format would you choose and why?**

Step 1. Pick a columnar compressed format - Choose Parquet or Avro with compression, with Parquet being preferred for columnar analytics because it reads only required columns and is space-efficient.

Step 2. Explain Parquet benefits - Parquet is columnar, so BigQuery reads fewer bytes for queries that touch a subset of columns, which lowers cost and speeds loads; it supports schema and efficient compression.

Step 3. Mention Avro as an alternative - Avro is row-based and supports schema evolution well and fast loads, so choose Avro if you need complex nested records with frequent schema changes and faster write throughput.

Step 4. Use compression to save storage and network - Store files compressed (Snappy, GZIP) to reduce GCS storage and lower egress/ingest time; Parquet with Snappy is a common, efficient combo.

Step 5. Partition files by load key - Organize GCS objects by date folders (for example gs://bucket/data/YYYY/MM/DD/) so you can run targeted load jobs and parallelize ingestion for daily 5TB slices.

Step 6. Use parallel load jobs and batch ingestion - Load data in parallel using BigQuery load jobs or Dataflow to avoid single-job bottlenecks; batch loading (file-based) is cheaper per-row than streaming.

Step 7. Validate schemas and enable schema auto-detection carefully - Define schemas or use Avro/Parquet embedded schema to avoid wrong types; if fields evolve, use schema evolution patterns (add nullable columns) to avoid failures.

Step 8. Optimize file sizes - Aim for many files sized between ~100MB and few GB each so you get good parallelism without too many tiny files that add overhead.

Step 9. Consider partitioned/clustered destination table - Load into a partitioned table (by date) and cluster on frequent filter columns so downstream queries are fast and cost-efficient.

Step 10. Automate and monitor the pipeline - Schedule daily jobs, add retries, validate row counts and checksums after load, and monitor load latency and cost so you can tune file sizes, parallelism, and schema handling over time.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 22. A Pub/Sub → BigQuery pipeline is dropping late-arriving events. How would you handle this?**

Step 1. Identify what "late" means for your use case - -define the maximum allowed lateness (for example 1 hour, 24 hours) so you can design retention and processing rules accordingly.

Step 2. Buffer events upstream - -change the pipeline to hold events temporarily (in Pub/Sub subscription ack-deadline extensions, a staging Pub/Sub topic, or Cloud Storage) so slightly-late messages can still be processed before final load.

Step 3. Use a durable staging layer - -write raw events to GCS or a staging BigQuery table first, then run idempotent batch loads into the final table; this lets you reprocess late events without losing data.

Step 4. Add event-time handling in Dataflow - -if you use Dataflow, configure windowing and allowed lateness (watermarks + allowedLateness) and side outputs for late elements so late events are either included or routed for special handling.

Step 5. Make BigQuery ingestion idempotent - -include a unique event ID and use INSERT with de-duplication logic (streaming insert with insertId or MERGE into partitioned table) so replays or late arrivals don't create duplicates.

Step 6. Accept reorder and reconcile with backfills - -provide a backfill job that re-reads staging data or GCS objects for time ranges affected by late arrivals and MERGEs them into the main table to repair historical partitions.

Step 7. Extend partition/window retention - -ensure partition expiration and streaming buffer window are long enough to accept late records; don't drop partitions too quickly.

Step 8. Monitor and alert on lateness - -add metrics/alerts for event latency, counts sent vs ingested, and quarantine rates so you know when late arrivals increase and can act.

Step 9. Communicate SLA and handle exceptions - -notify producers of lateness SLAs, provide a quarantine table for manual review, and define an operational playbook for reprocessing late batches.

Step 10. Iterate and automate - -after fixes, automate reconciliation (periodic replays) and keep observing late-event patterns to tune allowed lateness, buffering, and partition retention.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 23. A team wants near-real-time analytics with <10s latency. Which ingestion method would you suggest?**

Step 1. Recommend streaming ingestion (Pub/Sub → Dataflow → BigQuery) because it supports sub-second to second-level ingestion and processing, which is required for <10s end-to-end latency.

Step 2. Use Pub/Sub as the front door - -it provides durable, low-latency messaging and scales well for bursts; producers push events to Pub/Sub immediately.

Step 3. Process with a low-latency runner - -use Dataflow or a lightweight consumer that processes messages in streaming mode with small fixed windows, minimal transformations, and outputs to BigQuery quickly.

Step 4. Stream into BigQuery with best practices - -use tabledata.insertAll streaming inserts with insertIds for de-duplication, or write into a staging table and do fast MERGE if you need idempotency/atomicity.

Step 5. Minimize transformations in the hot path - -keep real-time pipeline logic small (parse, enrich with lightweight lookups cached in-memory) and defer heavy joins/aggregations to downstream micro-batch jobs to keep latency <10s.

Step 6. Use denormalized or pre-joined small lookup tables cached in the streaming worker - -avoid remote joins in the hot path; if lookups are needed, cache them or use embedded reference data.

Step 7. Optimize BigQuery for low-latency reads - -use partitioned tables, small-sketch summaries, or materialized views; consider BI Engine for in-memory acceleration of dashboards if sub-second reads are required.

Step 8. Monitor SLA and backpressure - -measure end-to-end latency from publish to queryable and add autoscaling and retries; implement dead-letter or quarantine topics for failures.

Step 9. Plan for consistency and duplicates - -handle at-least-once delivery with dedupe (insertId or unique keys + MERGE) and design idempotent consumers.

Step 10. Validate with load testing - -simulate expected event rates and latency constraints in staging to tune batching sizes, worker counts, and downstream write throughput to consistently meet <10s.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 24. Your CSV batch load is failing due to schema mismatch. How do you debug and fix it?**

Step 1. Read the exact BigQuery load error - -BigQuery error messages show which row/column failed and why (type mismatch, extra columns, bad date). Start by copying the error details.

Step 2. Inspect the CSV header and sample rows - -open a few failing rows and confirm the column order, delimiter, quoting, and encoding (UTF-8). Ensure header names and column count match the expected schema.

Step 3. Check field types vs CSV values - -compare CSV values with BigQuery column types (STRING, INT, TIMESTAMP). Look for bad formats (e.g., “N/A” in numeric column, malformed timestamps) and note offending rows.

Step 4. Try a small test load with schema detection off - -create a small sample CSV (10–50 rows) with the same structure and run the load job with an explicitly defined schema to reproduce and debug faster.

Step 5. Use CSV load options to handle quirks - -consider setting skipLeadingRows, allowJaggedRows, allowQuotedNewlines, or fieldDelimiter if files have variable structure; use nullMarker to map specific strings to NULL.

Step 6. Fix or cleanse data upstream - -correct bad values (convert “N/A” to empty, normalize date formats) or add a preprocessing step (Dataflow, Python script) to clean files before loading.

Step 7. Modify BigQuery schema carefully - -if data legitimately changed (new column added), update the destination schema to add a nullable column rather than REQUIRED, so historical rows still load.

Step 8. Validate with a dry-run or test loads - -once fixes are applied, run a test load on a portion of the data and verify row counts and types; iterate until clean.

Step 9. Automate checks and alerting - -add pre-load validation (schema checks, row-level validation) and fail fast with clear errors to reduce repeated failures.

Step 10. Document file contract and enforce upstream - -publish the agreed CSV schema (column order/types, timezone for timestamps) and have producers conform, or evolve the contract using versioning to avoid future mismatches.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**Question 25. Your BigQuery ingestion job is slow when loading small files. How would you improve performance?**

Step 1. Combine small files into larger files before loading - -small files cause overhead; coalesce many small objects into fewer files (for example 100MB–1GB each) in GCS to increase load throughput and parallelism efficiency.

Step 2. Use parallel load jobs - -split large daily loads into multiple parallel load jobs targeting different partitions or destination table slices so BigQuery can ingest concurrently.

Step 3. Prefer Avro/Parquet over many tiny CSVs - -compressed columnar formats and container formats with embedded schema reduce parsing overhead and speed loads compared to many small text files.

Step 4. Use streaming for tiny realtime events - -if you have many tiny event files arriving continuously, consider switching to Pub/Sub streaming ingestion to avoid repeated file-based load overhead.

Step 5. Batch small files into staging buckets - -implement a short micro-batching step that groups small files by time window into a staging file before kicking off the BigQuery load.

Step 6. Tune load job settings and retries - -ensure you use load jobs (not COPY) optimized for your file format and set sensible parallelism and retry logic to avoid serial bottlenecks.

Step 7. Load directly into partitioned tables - -target specific partitions when loading to avoid locking or scanning unrelated partitions and to enable parallel per-partition loads.

Step 8. Automate and monitor file sizes - -add pipeline checks that warn or auto-merge when average file sizes are below threshold, and monitor load latency and throughput.

Step 9. Use Dataflow for scalable ingestion and compaction - -Dataflow can read many small files, transform, and write larger files or directly stream into BigQuery efficiently.

Step 10. Iterate and measure - -after changes, measure load time, throughput, and costs; adjust target file size, parallelism, and format until ingestion performance meets SLA.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**Question 26. You are asked to calculate running totals of sales per customer. How would you solve this in BigQuery?**

Step 1. Clarify the business rule - -decide what “running total” means: cumulative sum by customer ordered by transaction time (event_time) or by invoice id; also decide tie-breakers (same timestamp) and how to handle late-arriving events.

Step 2. Use a window function for exact running totals - -the canonical SQL is SUM(sales_amount) OVER (PARTITION BY customer_id ORDER BY event_time ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) which produces a per-row cumulative total for each customer.

Step 3. Pick ORDER BY and frame carefully - -use ORDER BY event_time, transaction_id to ensure deterministic order when timestamps tie; prefer ROWS framing for precise row-by-row accumulation rather than RANGE when times repeat.

Step 4. Reduce input size first for performance - -if raw events are huge, pre-aggregate by day (or hour) per customer (customer_id, day, day_sales) and then run the cumulative sum on the much smaller daily aggregates to save memory and cost.

Step 5. Example query (daily pre-agg + running total) - -create daily sums: WITH daily AS (SELECT customer_id, DATE(event_time) AS day, SUM(amount) AS day_sales FROM events GROUP BY customer_id, day) then SELECT customer_id, day, SUM(day_sales) OVER (PARTITION BY customer_id ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total FROM daily;.

Step 6. Handle late-arriving events and re-computations - -if events can arrive late, either re-run the daily aggregation and re-compute the running totals for affected date ranges, or maintain idempotent merges into daily summary tables and incrementally recompute from the earliest changed day.

Step 7. Optimize storage and query speed - -store the pre-aggregated table partitioned by day and clustered by customer_id so incremental re-computes and per-customer scans are fast; consider materialized view for commonly-read running totals if query patterns are stable.

Step 8. Validate results and monitor - -compare a sample of exact-row-level cumulative sums vs the pre-aggregated approach to validate; monitor job runtime and bytes scanned and adjust partitioning/granularity (hour vs day) as needed.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**Question 27. A data scientist needs to compute 90th percentile latency from logs. Which SQL feature would you use?**

Step 1. Decide if approximate or exact is acceptable - -for very large log sets approximate percentiles (fast, cheap) are usually fine; if strict exactness is required and data is small enough, use exact analytic percentile functions.

Step 2. Use APPROX_QUANTILES for scale - -BigQuery's APPROX_QUANTILES(latency, 100) returns an array of 100 quantile cut points; the 90th percentile is quantiles[OFFSET(90)]. This is fast and memory-efficient.

Step 3. Example aggregate query (approx): SELECT (APPROX_QUANTILES(latency,100))[OFFSET(90)] AS p90 FROM dataset.logs WHERE event_date = '2025-09-12'; - -this gives the 90th percentile for that day.

Step 4. Use PERCENTILE_CONT / PERCENTILE_DISC for exact needs - -if you need exact percentile or windowed percentiles per group, PERCENTILE_CONT(latency, 0.9) OVER (PARTITION BY service) or aggregate SELECT PERCENTILE_CONT(latency, 0.9) FROM ... can be used, but exact functions are heavier.

Step 5. Compute per-group or sliding windows - -for per-service or per-region p90, use GROUP BY with APPROX_QUANTILES, e.g. SELECT service, (APPROX_QUANTILES(latency,100))[OFFSET(90)] FROM logs WHERE ... GROUP BY service;. For rolling windows, pre-aggregate into time buckets then compute p90 per bucket.

Step 6. Consider stability and sample size - -approximate algorithms work best with many samples; for small groups you may prefer exact percentile to avoid noise. Document the method and error tolerance.

Step 7. Store results for dashboards - -materialize daily/hourly p90s into a summary table so dashboards read small, stable rows instead of recalculating percentiles on raw logs repeatedly.

Step 8. Validate approximations - -occasionally compute exact percentiles on a sample window and compare with APPROX_QUANTILES to ensure approximation error is acceptable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 28. You need to transform nested JSON into relational format in BigQuery. How would you approach this?**

Step 1. Decide the target relational shape - -list which top-level columns you need and which nested arrays/objects must become separate tables (one-to-many) or repeated columns in the same table.

Step 2. Keep the raw JSON payload - -ingest and store the raw JSON (JSON type or STRING) so you can re-run parsing later if schema evolves; this is your single source-of-truth.

Step 3. Parse simple scalar fields into typed columns - -use JSON_EXTRACT_SCALAR (or JSON_VALUE / direct column access if JSON type) to map stable scalar fields into native BigQuery columns with correct types (STRING, INT64, TIMESTAMP). ExampleJSON_EXTRACT_SCALAR(payload, '$.user.id') AS user_id.

Step 4. UNNEST arrays into normalized rows - -for arrays (items, events) use CROSS JOIN UNNEST(JSON_EXTRACT_ARRAY(payload, '$.items')) AS item and then extract fields from item to produce one row per array element. If payload was loaded with RECORD/REPEATED schema you can CROSS JOIN UNNEST(items) AS item directly.

Step 5. Use SAFE parsing and handle missing fields - -use SAFE_CAST, NULLIF, and conditional logic to avoid errors on malformed data and to keep rows when fields are missing; mark rows with parsing errors for review.

Step 6. Build CTAS or staged tables - -transform in SQL into staging normalized tables: CREATE TABLE dataset.order_items AS SELECT order_id, JSON_EXTRACT_SCALAR(item, '$.id') AS item_id, ... FROM dataset.raw_orders, UNNEST(JSON_EXTRACT_ARRAY(raw_orders.payload, '$.items')) AS item;. This produces relational tables ready for joins.

Step 7. Automate with Dataflow for large or streaming workloads - -for heavy or streaming JSON loads, use Dataflow/Beam to parse, validate, and write to BigQuery tables with schemas, which handles schema evolution and retries more robustly.

Step 8. Add partitioning, clustering and indexes for performance - -partition the relational tables on event date and cluster on common join keys (order_id, customer_id) to speed queries and reduce cost.

Step 9. Test and document schema mapping - -create test cases for nested structures, document field paths and types, and provide example queries or views so analysts know how to read the transformed tables.

Step 10. Monitor and support evolution - -keep raw JSON copies, add automated alerts for new fields or schema drift, and have a plan to backfill or extend relational mappings when the incoming JSON schema changes.

© Shubham Wadekar

**All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar**

**Question 29. You are asked to join a table with an array column. How do you structure the query?**

Step 1. Understand the shape of data and the join goal - identify the table that has a repeated/ARRAY column (for example orders.items ARRAY<STRUCT<item_id, qty, price>>) and decide whether you need one row per array element (unnested) or to match at the array-level (exists/contains).

Step 2. Use UNNEST to turn array elements into rows when you need element-level joins - write FROM orders, UNNEST(items) AS item to produce one row per item so you can join item.item_id to the products table.

Step 3. Keep correct scoping with CROSS JOIN UNNEST for clarity - prefer CROSS JOIN UNNEST(orders.items) AS item (or , UNNEST(...) AS item) so it's obvious items are expanded per order row and you avoid accidental cross-joins across other tables.

Step 4. Join the unnested rows to the other table on the element key - for example LEFT JOIN products p ON item.item_id = p.product_id so each order-item row brings in product attributes; use LEFT JOIN to preserve orders with missing product matches.

Step 5. Re-aggregate if you need back to one row per parent - if the desired output is one row per order with aggregated item info, use GROUP BY orders.order_id and aggregate fields, e.g. SUM(item.qty * COALESCE(p.price, item.price)) AS order_total.

Step 6. Use EXISTS/IN for array-level matching when element detail not required - if you only need to check whether any array element matches a condition, use WHERE EXISTS(SELECT 1 FROM UNNEST(items) AS it WHERE it.item_id = 'X') to avoid full unnested joins.

Step 7. Watch for performance and reduce scanned data - select only needed fields, filter orders before UNNEST when possible (push predicates into the parent WHERE), and cluster/partition by common filter keys so UNNEST operates on a smaller dataset.

Step 8. Handle NULLs and empty arrays safely - use IFNULL(items, []) or guard logic because UNNEST on NULL differs from UNNEST on empty array; test with sample rows to ensure expected joins.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 30. Your team wants to reuse a custom function across multiple queries. What's the best approach?**

Step 1. Decide whether the function is pure SQL logic or needs procedural code - if it's reusable SQL logic (transformations, calculations), prefer a persistent SQL user-defined function (UDF) in BigQuery; if it needs external libraries, consider Cloud Functions/Dataflow.

Step 2. Create a Permanent SQL UDF in the dataset - write CREATE FUNCTION dataset.fn_name(arg TYPE) RETURNS TYPE AS ( /* SQL expression */ ); so any query in that project can call dataset.fn_name(...) without copy-pasting code.

Step 3. Use JavaScript UDFs only when SQL is insufficient - for complex parsing or algorithms not expressible easily in SQL, create a JavaScript UDF but be aware of performance and security limits.

Step 4. Package and document the function - add comments, examples, and expected input/output types in a central docs page so analysts use the function correctly and avoid accidental misuse.

Step 5. Manage versioning and changes safely - implement a versioned naming convention (for example utils.v1_calculate_score) and create new versions for breaking changes; deprecate old versions after consumers migrate.

Step 6. Use authorized views or wrapper functions for access control - if you need to restrict inputs or hide implementation details, provide an authorized view or a wrapper UDF that enforces validation and security.

Step 7. Test and benchmark - validate the UDF on representative data, measure cost and latency (UDFs can be slower), and optimize the logic or replace with native SQL alternatives if performance is poor.

Step 8. Consider stored procedures for multi-step logic - if the reuse requires multi-step stateful transformations, use BigQuery stored procedures (BEGIN...END) or move logic into an ETL job for better maintainability.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 31. A query that used to finish in 30s now takes 5 minutes. How do you troubleshoot it?**

Step 1. Reproduce and capture the exact query and execution metadata - run the same query in the UI or via API, then capture job id, query plan, bytes scanned, slot time, and timestamps from INFORMATION_SCHEMA.JOBS and job statistics.

Step 2. Compare recent successful run vs slow run - pull the earlier job details (when it ran in 30s) and compare stages, shuffle bytes, input rows, and how many slots were used to spot what changed (more bytes scanned, bigger shuffles, lower parallelism).

Step 3. Inspect SQL changes and parameter values - confirm the SQL text is identical, including literals and filters; check whether dashboard parameters or date ranges expanded (e.g., someone removed a partition filter) causing much larger scans.

Step 4. Check table statistics and partitioning health - see if partitions grew, data distribution changed, or clustering degraded (more fragmentation), which can increase scan/shuffle; use INFORMATION_SCHEMA.PARTITIONS and table metadata.

Step 5. Look at resource contention and slot availability - check slot reservation utilization and concurrent workloads at the slow time; other heavy jobs could be starving slots causing longer runtimes.

Step 6. Examine execution plan for costly stages - open the query plan to find large shuffle/merge/sort stages or disk spills that weren't present before; if spills increased, review joins/aggregations and data sizes.

Step 7. Test targeted optimizations - try adding a partition filter, limiting columns, pre-aggregating with a temp table, or rewriting joins to reduce shuffle; run small experiments to see which change restores performance.

Step 8. Check for external dependencies or recent schema changes - verify if referenced UDFs, views, or external tables (like federated sources) changed performance or introduced latency.

Step 9. Monitor system metrics and logs - review Cloud Monitoring for BigQuery slot usage, errors, and network/storage issues during the slowdown window; check audit logs for schema updates or DDLs.

Step 10. Apply fix and validate, then document root cause - after identifying cause (bad query, data growth, slot contention), implement the appropriate fix (query rewrite, repartition/cluster, increase slots, schedule competing jobs differently), re-run queries to confirm 30s behavior, and record the root cause and remediation to prevent recurrence.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 32. You want to track which users are running the most expensive queries. How would you monitor this?**

Step 1. Enable BigQuery audit logs in Cloud Logging to capture all query execution details including user identity, query text, and cost metadata.

Step 2. Query the INFORMATION_SCHEMA.JOBS_BY_USER or JOBS_BY_PROJECT views to see bytes processed, slot time, and execution time grouped by user.

Step 3. Aggregate costs per user by summing total_bytes_processed or total_slot_ms so you can identify top consumers over a chosen time range.

Step 4. Store the results in a monitoring dataset and schedule a recurring query (daily or weekly) that calculates cost metrics per user.

Step 5. Visualize in a dashboard (Looker Studio, Data Studio, or Grafana) showing top users, trends, and spikes in query cost.

Step 6. Configure Cloud Monitoring alerts if any single user exceeds defined thresholds (e.g., 10 TB scanned per day).

Step 7. Share regular usage/cost reports with team leads so they can review heavy queries and guide developers on optimization.

Step 8. Use authorized views or cost labels for stricter tracking if multiple teams share the same project, ensuring accountability per team or department.

**Question 33. A compliance team needs to audit all access to a sensitive dataset. How do you implement this in GCP?**

Step 1. Enable Cloud Audit Logs for BigQuery (Admin Activity, Data Access) so every read and write on the dataset is captured.

Step 2. Ensure that Data Access logs are enabled at the project or organization level, since these capture query-level access to specific datasets and tables.

Step 3. Route audit logs to a central Cloud Logging sink dedicated for compliance, and export them to BigQuery or GCS for long-term retention.

Step 4. Create an audit dataset/table that stores structured access logs including user identity, timestamp, dataset/table name, and action type.

Step 5. Build queries or dashboards for compliance officers to filter by user, dataset, or time to see who accessed sensitive data.

Step 6. Apply IAM least privilege on the sensitive dataset so only authorized users can query it; enforce that all queries are captured through logging.

Step 7. Set retention policies and archive logs according to compliance requirements (e.g., 7 years).

Step 8. Monitor and alert on suspicious access patterns (e.g., new user accessing sensitive dataset, large downloads) using Cloud Monitoring or Security Command Center.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 34. A job is stuck in “RUNNING” state for a long time. What steps would you take?**

Step 1. Open the BigQuery job details in the console or query INFORMATION_SCHEMA.JOBS to confirm the job id, start time, and current state.

Step 2. Check the execution plan to see if a stage is stuck waiting on resources (slots) or if it is processing an unusually large shuffle or scan.

Step 3. Review table sizes, partition filters, and joins in the SQL to see if the query is accidentally scanning too much data or doing a cross join.

Step 4. Check slot usage and reservations to confirm if the project or reservation has enough slots; jobs may hang in “RUNNING” while waiting for resources.

Step 5. Look at Cloud Monitoring for BigQuery slot utilization and job concurrency to see if other heavy jobs are blocking this one.

Step 6. If the query depends on external sources (federated tables, external functions, remote UDFs), verify that those systems are responsive.

Step 7. Consider canceling the job if it is consuming excessive resources or blocking production workloads; use bq cancel <job_id> or console.

Step 8. Optimize the query by adding partition filters, reducing columns, or breaking into smaller stages, then rerun with a smaller test dataset first.

Step 9. If jobs are consistently stuck, consider increasing slot capacity, scheduling heavy queries in off-peak times, or redesigning the data model.

Step 10. Document the root cause (query issue, slot contention, external dependency) and apply fixes in both pipeline and resource planning to prevent recurrence.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 35. Your team wants to identify queries that cost more than $100. How do you detect them?**

Step 1. Decide the cost threshold in bytes - -convert $100 into bytes scanned using your on-demand price (for example if $5 per TB scanned, $100 → 20 TB) so you can detect by bytes rather than dollars.

Step 2. Query INFORMATION_SCHEMA.JOBS_BY_PROJECT (or JOBS_BY_USER) to find recent queries and their total_bytes_processed and total_slot_ms - -these fields let you estimate cost per job.

Step 3. Create a scheduled query that runs every hour/day which selects jobs where total_bytes_processed exceeds the bytes threshold and writes those rows into a query_cost_alerts table for tracking.

Step 4. Add useful metadata to the alert table - -store job_id, user_email, query_text (or first N chars), start_time, end_time, bytes_processed, estimated_dollars, and labels so owners can be contacted.

Step 5. Build a dashboard (Looker Studio or BigQuery UI) that shows queries over the threshold grouped by user, project, or query text to spot repeat offenders and trends.

Step 6. Wire alerts: create Cloud Monitoring alerts or a Cloud Function triggered by a scheduled check that emails or posts to Slack when a new query exceeds the threshold.

Step 7. Enforce preventive controls - -set default maximumBytesBilled on user jobs or apply a policy requiring partition filters; use reservations/quotas to block runaway queries if needed.

Step 8. Automate remediation steps - -when threshold breaches are detected, automatically notify the user and optionally create a ticket or flag their queries for review; keep the alerting workflow auditable.

Step 9. Iterate and refine the threshold - -compare estimated dollars vs actual billing over a few weeks to tune the bytes→dollars conversion and reduce false positives.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 36. Your BI dashboard runs the same aggregation query 100 times a day. How do you optimize this with materialized views?**

Step 1. Confirm the aggregation is stable and repetitive - -verify the exact SQL used by the dashboard is the same each run (same groupings, filters, and time window) because materialized views accelerate consistent queries.

Step 2. Create a materialized view that precomputes the aggregation - -use CREATE MATERIALIZED VIEW dataset.mv_name AS SELECT ... with the same aggregation and GROUP BY used by the dashboard so BigQuery maintains it incrementally.

Step 3. Ensure the base table(s) have appropriate partitioning - -materialized views perform best when the underlying tables are partitioned (for example by date) so incremental maintenance touches small partitions.

Step 4. Point the dashboard to the materialized view - -change the dashboard data source to read from the MV instead of running the full aggregation query; this yields much faster reads and lower cost.

Step 5. Understand freshness guarantees - -materialized views are incrementally updated but may not be instant; check the MV refresh behavior and ensure its latency meets the dashboard SLA.

Step 6. Monitor MV maintenance cost and effectiveness - -track how often the MV is updated, bytes processed during maintenance, and query hit rate (how many dashboard queries used the MV) to ensure net cost/performance benefit.

Step 7. Tune the MV or create additional MVs if dashboard has multiple distinct queries - -for different common filters or groupings, create separate materialized views or narrow summary tables to maximize cacheability.

Step 8. Fall back to summary tables for complex cases - -if the aggregation is too complex for an MV or MV maintenance cost is high, schedule a daily/hourly summary table populated by ETL that the dashboard reads instead.

Step 9. Add access controls and versioning - -use authorized views or dataset permissions to safely swap MVs into dashboards and keep a migration path if you change the aggregation logic.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 37. A stakeholder complains cached query results are outdated. How do you explain caching vs materialized views?**

Step 1. Explain what query caching is - -cached results store a previous query's output for a short time; if the exact same SQL runs and the underlying tables haven't changed, BigQuery returns the cached result instantly.

Step 2. Explain cache invalidation rules - -cache is invalidated when underlying table data changes (writes, loads, updates) or when the query text differs even slightly; also cached results are temporary and not guaranteed for long-term use.

Step 3. Describe materialized views behavior - -materialized views store a maintained, precomputed result set that BigQuery incrementally updates as base tables change; they are designed for repeated queries and provide faster, more predictable reads.

Step 4. Compare freshness guarantees - -caching returns the last known result only while valid and can become outdated as soon as data changes; materialized views are updated incrementally and provide a controlled, often fresher snapshot for repeated queries.

Step 5. Compare control and intent - -caching is passive and automatic (no setup) but ephemeral; materialized views are an explicit design choice that you create and maintain for a specific query pattern.

Step 6. Compare cost and performance trade-offs - -cache gives near-zero cost/latency for identical repeated queries but is unreliable when data changes; materialized views incur maintenance cost on updates but reduce repeated aggregation cost and give consistent performance.

Step 7. Recommend when to use each - -use query caching for ad-hoc identical queries and quick wins; use materialized views when dashboards run the same aggregation frequently and need predictable freshness and performance.

Step 8. Give a practical policy - -if stakeholders need guaranteed up-to-date dashboards, use materialized views (or scheduled ETL) and set refresh windows; if near-instant but occasionally stale results are acceptable, rely on caching but communicate its limits.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 38. You have a materialized view on a partitioned table. How do you ensure it updates incrementally?**

Step 1. Confirm the MV is supported for incremental refresh - -ensure the materialized view is a valid incremental MV (simple aggregation/group by or deterministic SELECT) because BigQuery only incrementally maintains MVs that meet its restrictions.

Step 2. Partition the base table on a stable time key - -use a clear partition column (e.g., event_date) so BigQuery can determine which partitions changed and apply incremental maintenance only for those partitions.

Step 3. Design the MV to reference the partition column (directly or via predicates) - -include the partition key in the MV's grouping/filters when appropriate so incremental updates map cleanly to partition changes.

Step 4. Avoid non-deterministic expressions and unsupported features - -remove functions like RAND(), CURRENT_TIMESTAMP() or cross-database references inside the MV SQL because they prevent incremental maintenance.

Step 5. Keep the MV's query relatively simple and stable - -simple aggregations and joins on partitioned, well-typed columns let BigQuery track deltas and update only changed partitions rather than recomputing the whole view.

Step 6. Monitor MV maintenance activity and freshness - -use INFORMATION_SCHEMA and job history to see how often the MV is being updated and how many bytes are processed so you can verify incremental behavior.

Step 7. Tune base-table ingestion to help incremental refresh - -write/append data into partitioned batches (not tiny per-row updates) so BigQuery can efficiently apply incremental changes; avoid frequent schema changes that would force full recompute.

Step 8. If incremental updates still trigger full refresh, refactor the MV or use scheduled summary tables - -convert the MV into an ETL-maintained summary with partitioned writes if BigQuery's incremental maintenance cannot be achieved for your query shape.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 39. Your query doesn't use cached results even though it should. What could be the reason?**

Step 1. The underlying table changed since the cached result - -any write, load, or DML to a table used by the query invalidates the cache, so the same query won't use cache after data change.

Step 2. The query text differs even slightly - -whitespace, comments, literals, or parameter ordering changes make BigQuery treat it as a different query and prevent cache hits.

Step 3. The query used non-deterministic functions or volatile expressions - -functions like CURRENT_TIMESTAMP(), RAND(), or any nondeterministic UDF prevent caching because results can differ each run.

Step 4. The query referenced external or federated data sources - -queries over external tables (Cloud Storage, Cloud SQL) or federated sources are typically not cached because their state may change outside BigQuery.

Step 5. You disabled query caching explicitly - -a job-level or client setting (useQueryCache=false) will prevent cache usage; check your client tool or job config.

Step 6. The cached result expired or was evicted - -BigQuery does not guarantee indefinite caching; if the cached result was evicted or expired, subsequent identical queries won't hit cache.

Step 7. Authorization or access differences - -if the query runs under a different service account or user that lacks access to the cached result metadata, BigQuery may not reuse the cache.

Step 8. The query used temporary tables or session-specific objects - -caches typically won't be used when temporary resources or session-scoped tables are involved because they change per session.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 40. Your BI team needs sub-second dashboard performance. How would you combine BI Engine and materialized views?**

Step 1. Identify the hot, repetitive queries - -find which dashboard tiles run the same aggregation frequently and confirm they are eligible for materialized views (stable SQL) so you can precompute results.

Step 2. Create materialized views for those aggregations - -let BigQuery maintain the pre-aggregated results incrementally so the dashboard reads a small, precomputed dataset instead of scanning raw tables.

Step 3. Reserve BI Engine capacity sized for concurrency and dataset size - -provision BI Engine memory that fits the working set of the MV results and concurrent dashboard users so queries can be served from memory.

Step 4. Point BI dashboards at the materialized views - -configure the BI tool to query the MV (not raw tables) so BI Engine can cache MV data in-memory and serve sub-second responses.

Step 5. Tune MV granularity for BI Engine fit - -design MVs with the right aggregation grain (daily/hourly, by region) so the materialized result set is compact enough to fit in BI Engine memory but still answers dashboard needs.

Step 6. Monitor BI Engine cache hit rates and MV maintenance cost - -check how often BI Engine serves from memory and measure MV incremental update overhead; adjust MV definitions or BI Engine size if hit rate or freshness is suboptimal.

Step 7. Handle freshness vs latency trade-offs - -set expectations: BI Engine + MV gives sub-second reads for maintained aggregates, but there's a small maintenance lag; if absolute real-time is required, combine with lightweight streaming summaries for the hottest metrics.

Step 8. Iterate and scale - -add more targeted MVs for other heavy tiles, increase BI Engine capacity if needed, and keep monitoring dashboard latency and MV update throughput to preserve sub-second performance.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 41. Your team wants to query Parquet files directly from GCS. What's the trade-off vs loading into BigQuery?**

Step 1. Cost - Querying Parquet in GCS (external table / federated query) avoids storage cost inside BigQuery but may cost more per-query because external reads can be slower and less optimized; loading into BigQuery charges storage but usually lowers query bytes scanned and cost per query over time.

Step 2. Performance - External Parquet reads are I/O bound and can be slower (higher latency) because BigQuery can't fully optimize layout or use column pruning/metadata as efficiently; loaded tables are served from BigQuery's internal storage and often run faster.

Step 3. Features and optimizations - Loaded tables support partitioning, clustering, materialized views, BI Engine acceleration, and slot-based performance tuning; external tables have limited support for some of these features and may miss incremental maintenance benefits.

Step 4. Freshness and workflow - External tables are great if you need immediate visibility of files dropped into GCS (no load job), but for repeatable analytics you may prefer loading and compacting data into partitioned tables to improve query stability and reduce scanning.

Step 5. Schema evolution and validation - Parquet carries schema, but federated queries can face type or nested-structure issues; loading gives you a controlled schema migration process, admission checks, and the ability to backfill or clean data before analysts consume it.

Step 6. Parallelism and small-file behavior - BigQuery load jobs can compact many small Parquet files into efficient storage; querying many small external files can be inefficient. If your pipeline produces many tiny files, loading and compaction is usually better.

Step 7. Cost predictability and repeated queries - If you run the same queries often, loading into BigQuery (partitioned/clustered) reduces repeated read costs and gives more predictable performance and cost; external tables are better for ad-hoc, infrequent inspections or ETL staging.

Step 8. Operational complexity - External tables simplify quick access but push data correctness and lifecycle to GCS; loading adds a controlled ETL step (monitoring, retries, validation) but gives better governance and query reliability for production analytics.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 42. A business user wants to query a Google Sheet alongside BigQuery tables. How do you enable this?**

Step 1. Use BigQuery federated query for Google Sheets - Create an external table in BigQuery that references the Google Sheet (via CONNECTOR or the Sheets external table feature) so SQL can join the sheet data with BigQuery tables.

Step 2. Prepare the Google Sheet - Ensure the sheet has a header row, consistent column types, and a shared permission that allows the BigQuery service account (or the user) to read the sheet. Validate small samples first.

Step 3. Create the external table - In BigQuery create a table with EXTERNAL_DATA_CONFIGURATION pointing to the Google Sheet URL, or use the BigQuery UI "Create external table" and choose Google Sheets as the source; define or autodetect the schema.

Step 4. Join in SQL like any table - Write queries that join the external sheet table to internal BigQuery tables (for example SELECT * FROM bq_table t JOIN dataset.sheet_ext s ON t.key = s.key) - -treat it as read-only input.

Step 5. Consider performance and caching - Google Sheets external tables are slow for large reads; for repeated or larger joins, copy the sheet data into a staging BigQuery table (via scheduled load) so joins run fast and reliably.

Step 6. Automate refresh if needed - If the sheet changes frequently and you need up-to-date views, schedule a periodic job to load the sheet into a partitioned/staging table in BigQuery or use the Sheets connector with an agreed refresh cadence.

Step 7. Manage access and security - Control who can create external tables and ensure the sheet's sharing settings do not expose sensitive data; use authorized views or IAM to restrict downstream access.

Step 8. Document the contract for business users - Tell users expected formats, column names, and refresh windows, and provide a simple script or scheduled job to sync their sheet into BigQuery for production workflows.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 43. You need to query Cloud SQL data in BigQuery. What's the best method?**

Step 1. Choose pattern based on latency and volume - For low-latency, small-volume needs use BigQuery federated queries via the Cloud SQL connector; for larger or production analytics, replicate data from Cloud SQL into BigQuery (ETL/CDC) for performance and scale.

Step 2. Use the BigQuery Cloud SQL federated connector for ad-hoc queries - This connector lets you query Cloud SQL (MySQL/Postgres) directly from BigQuery without copying data, useful for occasional joins or quick checks. Note it's slower and not suited for heavy workloads.

Step 3. For production analytics prefer replication/ETL - Use Dataflow, Database Migration Service, or a CDC tool (Debezium, Fivetran, Stitch) to stream or batch-change-data-capture from Cloud SQL into partitioned BigQuery tables so analytics run fast and reliably.

Step 4. Design the replicated schema and cadence - Decide whether to do near-real-time CDC (keeping BigQuery up-to-date) or scheduled full/incremental loads (nightly/daily); choose partitioning keys (by date) and clustering for query patterns.

Step 5. Ensure idempotency and deduplication - When using streaming/CDC, include a primary key and ordering metadata so you can MERGE into BigQuery safely and avoid duplicates on retries.

Step 6. Handle schema changes and migrations - Put schema evolution logic in the ETL so new Cloud SQL columns are added as nullable in BigQuery and backfills are performed when needed; avoid breaking consumers.

Step 7. Monitor latency, data drift, and costs - Track replication lag, row counts, and errors; for federated queries watch latency and cost; for replication watch slot usage and storage cost in BigQuery.

Step 8. Pick the simplest path for the use-case - If the requirement is one-off or exploratory, use federated queries; if dashboards or heavy analysis depend on the data, invest in a replicated pipeline to BigQuery for scale, cost-efficiency, and stability.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 44. Your federated query on GCS is too slow. What optimizations can you apply?**

Step 1. Profile the query and files - check the query plan and which files are read; measure number of files, file sizes, and bytes read so you know whether slowness comes from many tiny files, wide scans, or complex SQL.

Step 2. Compact small files before querying - merge many small objects into fewer larger files (hundreds of MBs to low-GBs) so the federated reader has less per-file overhead and can read in fewer, more efficient streams.

Step 3. Use columnar formats (Parquet/ORC) and compression - convert CSV/JSON to Parquet or ORC with Snappy/GZIP so federated reads only the needed columns and reduce I/O significantly.

Step 4. Push filters and projections early - rewrite queries to filter on partition-like prefixes (gs://bucket/path/date=2025-09-12/...) and select only required columns so less data is transferred from GCS.

Step 5. Partition files by query key (date/hour/region) - organize GCS objects into logical folders by the most common filter (date/region) so queries can target small subsets rather than scanning everything.

Step 6. Cache or load hot data into BigQuery - for frequently-queried datasets, run an initial load into partitioned BigQuery tables (or create scheduled loads) so repeated queries run much faster and cheaper than federated reads.

Step 7. Avoid expensive operations over federated data - move heavy joins, sorts, and aggregations into BigQuery after loading; perform only light filtering or previewing on GCS data.

Step 8. Use Dataflow to pre-process or read efficiently - if conversion or enrichment is needed, use Dataflow to transform/compact files and write optimized files or directly stream into BigQuery for faster downstream queries.

Step 9. Tune client and parallelism - run multiple federated queries in parallel carefully (avoid overwhelming GCS), and where possible parallelize reads by splitting work across file groups and using BigQuery load jobs for large-scale parallel ingestion.

Step 10. Monitor and iterate - track read throughput, query latency, and bytes read after each change; continue adjusting file size, format, and partitioning until federated query performance meets your needs.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 45. A project requires joining Bigtable data with BigQuery data. How would you design this pipeline?**

Step 1. Choose join strategy based on latency and volume - for infrequent or small joins use a lookup pattern (read BigQuery rows and call Bigtable for keys) or federated/side-input; for large or repeated analytic joins, replicate Bigtable data into BigQuery.

Step 2. Replicate Bigtable to BigQuery for analytics - build a CDC or scheduled export pipeline (Dataflow with Cloud Bigtable connector or custom job) that exports relevant Bigtable rows to partitioned BigQuery tables so joins run at scale and with BigQuery optimizations.

Step 3. Transform and shape the replicated data - during replication, project only needed columns, normalize keys and types, and add timestamps/metadata so BigQuery tables are query-friendly and partitionable (for example by event_date).

Step 4. Use partitioning and clustering on replicated tables - partition by date or logical time and cluster on join keys used with BigQuery tables to speed join pruning and reduce shuffle.

Step 5. For low-latency needs keep an online lookup layer - if application needs per-request enrichment, keep Bigtable as primary low-latency store and implement an online lookup (app-level join) that fetches Bigtable rows by key and merges with BigQuery-derived data served via APIs or materialized extracts.

Step 6. Implement idempotent, incremental updates - use timestamps, change streams, or row versions in Bigtable exports; in BigQuery use MERGE to apply incremental changes and avoid duplicates when replicating.

Step 7. Optimize join queries in BigQuery - prefer joins on equality keys, filter early, pre-aggregate or pre-join smaller dimension tables, and use broadcast joins by ensuring the smaller side is truly small or preloaded as a lookup table.

Step 8. Consider hybrid approaches for cost/performance - keep very large raw Bigtable content in Bigtable and replicate summarized or dimension slices to BigQuery for analytics; only replicate what analytics needs.

Step 9. Monitor freshness, latency and cost - track replication lag, row counts, and query performance; alert on replication failures and tune batch size or streaming cadence to meet SLAs.

Step 10. Document access patterns and ownership - define who owns replication, schema mapping, and transformations; provide sample queries and views so analysts can reliably join Bigtable-derived tables with BigQuery datasets.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 46. A table was accidentally deleted yesterday. How would you recover it in BigQuery?**

Step 1. Check if table was deleted or dataset dropped and note the deletion time - use audit logs (Activity in Cloud Console or Cloud Logging) to find the exact timestamp and job/user that performed the deletion.

Step 2. Use BigQuery's time travel (table snapshot) - if within 7 days of deletion, restore the table by querying the table snapshot using the FOR SYSTEM_TIME AS OF clause against the table's previous state or by copying the table from the snapshot: CREATE TABLE dataset.restored_table AS SELECT * FROM dataset.deleted_table FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL X DAY); replacing X with appropriate hours/days.

Step 3. If the table name no longer exists, reference it by its fully-qualified historical snapshot - use project.dataset.__TABLES__ metadata or the restore pattern; you can also use the deleted table's resource name with FOR SYSTEM_TIME AS OF if you captured the exact deletion timestamp.

Step 4. Restore from a backup export if time travel not available - if the deletion is older than time-travel window or time-travel was disabled, restore from exported backups in GCS (Avro/Parquet/CSV) if you have scheduled exports.

Step 5. Recover from audit logs or job history - if you have scheduled exports or ETL job outputs, replay the latest successful export or re-run the ETL to re-create the table up to the last known good state.

Step 6. Use dataset-level/table-level snapshots if previously created - if you or your team had taken manual snapshots (copy tables, backups), copy the snapshot table back into production with CREATE TABLE ... AS SELECT * FROM backup_table.

Step 7. Validate restored data - compare row counts, checksums, and a few sample rows with expected values or previous reports to ensure the restore is correct before pointing consumers back to the table.

Step 8. Recreate dependent objects and permissions - if views, routines, or IAM bindings were tied to the deleted table, recreate or rebind them as needed and test downstream jobs/dashboards.

Step 9. Implement protections to avoid repeat incidents - enable dataset-level IAM controls, restrict deletion permissions, enable automated backups/export, and consider labels/alerts for destructive DDL operations.

Step 10. Document the incident and recovery steps - record deletion cause, restore method, time taken, and lessons learned; update runbooks so future deletions are recovered faster.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 47. Your legal team requires 90-day retention for rollback. How would you configure BigQuery datasets?**

Step 1. Set dataset-level default table expiration - -configure the dataset so that all new tables automatically have an expiration of 90 days unless overridden; this enforces retention without manual action.

Step 2. Use table-level expiration for critical tables - -for specific tables, set expiration_timestamp to 90 days from creation so rows are automatically deleted after the retention window.

Step 3. Use partition expiration for time-partitioned tables - -configure partition expiration to 90 days so old partitions drop automatically, retaining only the rolling 90-day window.

Step 4. Document and enforce via governance - -publish the retention policy so engineers know data older than 90 days will be unavailable; enforce via dataset IAM and default expiration settings.

Step 5. For rollback needs, rely on time travel snapshots - -BigQuery supports 7-day time travel; for 90 days, schedule daily table snapshots into an audit/backup dataset that has its own 90-day expiration.

Step 6. Export to GCS if longer history is needed - -configure scheduled exports to GCS with lifecycle policies set to 90 days for rollback copies.

Step 7. Monitor compliance - -query INFORMATION_SCHEMA.TABLES to check expiration settings regularly and alert if tables are missing 90-day limits.

**Question 48. A table grows very large over time. How would you manage retention and expiration?**

Step 1. Analyze growth and access pattern - -determine how long data is actually used (e.g., last 90 days for reports, 1 year for compliance) so you can set expiration accordingly.

Step 2. Partition the table by ingestion date or event date - -this lets you drop or expire old partitions instead of managing the full table, reducing growth over time.

Step 3. Configure partition expiration - -set partition expiration (for example 90 days) so old data automatically deletes and storage is managed without manual cleanup.

Step 4. Use table expiration if not partitioned - -set a table-level expiration timestamp to ensure the table is deleted after a certain age; use this only if full-table lifecycle fits the business case.

Step 5. Export old data to cheaper storage - -if compliance requires keeping older data, export expired partitions to GCS in Parquet/Avro for archival, then delete them from BigQuery.

Step 6. Compact data to reduce size - -periodically rewrite or merge fragmented partitions and store in compressed formats to minimize long-term growth.

Step 7. Monitor size and cost - -use INFORMATION_SCHEMA and billing export tables to track table growth and ensure retention settings are working as planned.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 49. You need a point-in-time snapshot of a table for auditing. How would you do it?**

Step 1. Use time travel queries - -BigQuery allows you to query a table FOR SYSTEM_TIME AS OF TIMESTAMP 'YYYY-MM-DD HH:MM:SS' to see its state at that point in time (within 7 days).

Step 2. Create a copy from the snapshot - -run CREATE TABLE audit.snapshot_table AS SELECT * FROM dataset.table FOR SYSTEM_TIME AS OF TIMESTAMP ... to materialize the point-in-time copy.

Step 3. For long-term retention beyond 7 days - -schedule regular snapshots into an audit dataset, keeping copies at required intervals (daily/weekly/monthly) for compliance.

Step 4. Store snapshots in an audit-only dataset - -separate storage for auditing ensures snapshots are immutable and access-controlled.

Step 5. Apply IAM restrictions - -limit access to snapshot datasets to compliance and audit teams only, ensuring audit integrity.

Step 6. Document and monitor - -record snapshot creation in a log or metadata table and verify snapshot retention policies meet audit requirements.

**Question 50. Your project costs are spiking because of long-term storage. How would you address this?**

Step 1. Identify which tables are in long-term storage - -query INFORMATION_SCHEMA.TABLE_STORAGE to see which tables are older than 90 days and contributing to long-term storage cost.

Step 2. Review actual usage - -check query logs to see if old data is accessed; if rarely used, move it out of BigQuery hot storage.

Step 3. Apply partition expiration - -for partitioned tables, configure expiration (e.g., 90 or 180 days) so partitions drop automatically, reducing accumulation of unused old data.

Step 4. Export old partitions to cheaper storage - -archive historical data to GCS in compressed Parquet/Avro and delete from BigQuery; GCS Nearline or Coldline can reduce costs significantly.

Step 5. Compact and deduplicate data - -rewrite fragmented tables or merge small partitions into optimized storage to reduce overhead.

Step 6. Delete unused datasets - -remove tables or datasets no longer required by compliance or business; set dataset-level expiration for future prevention.

Step 7. Monitor storage growth - -schedule regular checks on dataset sizes and set alerts if storage exceeds thresholds so you can take proactive action.

Step 8. Educate teams and enforce policies - -ensure engineers understand storage cost implications and enforce retention/expiration policies at dataset level to prevent uncontrolled growth.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 51. Your query is failing due to shuffle size limit. How do you fix it?**

Step 1. Confirm the failure reason and capture query details - -check the job's execution plan and error message in BigQuery job details to verify it's a shuffle-size limit (large stage shuffle, worker memory exceeded) and note which stage and which tables are involved.

Step 2. Reduce data before the shuffle - -push filters and projections earlier (filter rows and select only needed columns in subqueries) so each stage moves far less data into the shuffle.

Step 3. Pre-aggregate or pre-filter large inputs - -aggregate (GROUP BY) or filter in a prior step and write results to a temp/partitioned table so the join/aggregation stage handles much smaller payloads.

Step 4. Replace heavy joins with broadcast when safe - -if one side is small (see Question 52), materialize it or load it into a temporary table and use it as a broadcast/lookup to avoid a large shuffle.

Step 5. Break the query into smaller stages - -split the query into multiple steps (CTE → CREATE TABLE AS SELECT → final join) so each stage fits memory limits and BigQuery can handle smaller shuffles.

Step 6. Use partitioning and clustering to limit shuffle scope - -ensure tables are partitioned on query filters and clustered on join keys so BigQuery prunes data early and shuffles less.

Step 7. Use approximate functions or sampling when exactness isn't required - -approximations (APPROX_*) or sampling reduce the amount of data to aggregate and therefore reduce shuffle.

Step 8. Check for data skew and rebalance keys - -if a few keys are huge, they cause hotspots; consider re-bucketing (hashing) or pre-aggregating those keys separately to avoid skewed worker load.

Step 9. If resource-bound, consider more slots or reservation tuning - -increasing slot capacity or adjusting reservations can reduce contention, but this is complementary - -the primary fixes are reducing shuffle size and rewriting query shape.

Step 10. Test and monitor the fix - -rerun the job, inspect the new plan and shuffle bytes, and iterate until shuffle stages complete without hitting limits.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 52. A join between two massive tables is slow. How do you decide between broadcast and shuffle join?**

Step 1. Measure table sizes and cardinality - -get the byte size and row counts for both tables (or for the filtered subsets used in the query); broadcast is viable only if one side is small enough to replicate to all workers.

Step 2. Rule of thumb: broadcast the much smaller side - -if one side (after filtering) is small (typically low hundreds of MBs to a few GBs depending on your environment), broadcast it to avoid expensive shuffles; otherwise use a shuffle join.

Step 3. Consider join key distribution and skew - -if the small table is small but keys are extremely skewed, broadcasting may still cause hotspots; prefer pre-aggregation or rebucketing in that case.

Step 4. Evaluate query frequency and repeated usage - -if the small side is reused often, materialize it as a compact lookup table (or temp table) so broadcasts are cheaper and stable.

Step 5. Assess memory and slot constraints - -broadcasting multiplies memory usage on each worker; ensure slot/memory capacity supports replication. If not, use shuffle joins with query rewrites.

Step 6. Think about update patterns - -if the small table changes frequently, broadcasting may add maintenance cost; for stable dimension data broadcasting is easier.

Step 7. Try hybrid tactics - -pre-aggregate the large table by join key to shrink it, or partition the join by date so each join touches smaller slices and can be parallelized efficiently.

Step 8. Test both approaches at scale - -run representative queries (or use explain/plan) to compare shuffle bytes, memory use, and latencies; pick the method that gives acceptable latency and cost.

Step 9. Document the decision and fallback - -record thresholds and patterns for future queries and provide a fallback (split into stages or increase slots) if data volume grows beyond the broadcast threshold.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 53. A query is spilling to disk frequently. How would you optimize it?**

Step 1. Identify which operations spill - -inspect the query execution plan and job statistics to see whether joins, sorts, or aggregations are causing spills and which stage(s) are affected.

Step 2. Push down filters and reduce columns early - -apply WHERE filters and select only necessary columns before heavy joins/aggregations so each stage works on less data and needs less memory.

Step 3. Pre-aggregate or materialize intermediate results - -compute GROUP BY or dedupe steps in a prior job and write a smaller temp table, then run the heavy operations on that compacted dataset.

Step 4. Rewrite joins to be more memory-friendly - -use broadcast joins for small lookup sides, avoid cross joins, and ensure join keys are simple and typed correctly to prevent extra memory overhead.

Step 5. Avoid expensive global sorts and large DISTINCTs - -if you only need top-N, use ORDER BY ... LIMIT N; for deduplication prefer ROW_NUMBER() with partitioning rather than SELECT DISTINCT on billions of rows.

Step 6. Use partitioning and clustering on base tables - -partition by time or logical slices and cluster on join/filter keys so scans and shuffles are smaller and localized, reducing spill risk.

Step 7. Increase available resources if needed - -if query shape is already optimal but still spills, allocate more slots or reserve capacity so workers have more memory to avoid disk spilling.

Step 8. Break the job into smaller tasks - -split the query into sequential stages (write intermediate tables), which reduces peak memory and prevents a single large stage from spilling.

Step 9. Use approximate functions when acceptable - -approximate aggregations or sketches reduce memory pressure and eliminate large sorts/merges that cause spills.

Step 10. Monitor and iterate - -after changes, verify reduced spill metrics, lower shuffle bytes, and improved runtimes; keep tuning query shape, partitioning, and resource allocation until spills are rare.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 54. You want to explain query stages to a junior engineer. How do you describe them simply?**

Step 1. Think of a query as a pipeline broken into steps, where each step does part of the work like filtering, joining, or aggregating.

Step 2. Each stage takes the data from the previous stage, does its job (for example, filtering rows or computing sums), and passes the result to the next stage.

Step 3. BigQuery runs stages in parallel across many workers, so multiple chunks of data are processed at the same time to make it fast.

Step 4. Some stages are "local" (processing data already in one worker, like filtering) while others require "shuffle" (moving rows across workers, like joining or grouping by a key).

Step 5. Shuffle-heavy stages are usually the slowest, because BigQuery needs to move large amounts of data between workers before continuing.

Step 6. You can see stages in the query plan in the BigQuery console - -it shows boxes (stages) connected by arrows, each box summarizing what kind of work it did and how much data it processed.

Step 7. Optimizing queries often means reducing the size of data that enters expensive stages (joins, sorts, group-bys) so fewer rows need to be shuffled.

Step 8. Tell the junior engineer: "Think of query stages like an assembly line - -each worker does one job, then passes things on. If one stage is too big or slow, it delays the whole line."

**Question 55. Why does a query run fast for small data but very slow for large datasets?**

Step 1. On small data, BigQuery can process everything quickly in memory with little or no shuffling, so queries finish in seconds.

Step 2. As data grows, queries scan far more bytes, so the read step alone takes longer and costs more.

Step 3. Larger datasets mean more rows must be shuffled between workers for joins, group-bys, or sorts, which increases network and disk usage.

Step 4. Memory limits are hit with large data, so BigQuery may spill intermediate results to disk, which is much slower than in-memory.

Step 5. Data skew becomes visible at scale - -some keys may have millions of rows while others have few, creating "hotspots" that slow down the whole query.

Step 6. Poor schema design (no partitioning/clustering) hurts at scale - -without pruning, queries scan the full dataset instead of just relevant slices.

Step 7. Repeated aggregations or wide SELECT * queries on large tables multiply costs, while on small datasets this overhead isn't noticeable.

Step 8. In summary: small data queries stay in memory and avoid heavy movement, while large data queries involve more I/O, network shuffle, and disk spilling, making them much slower unless optimized.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 56. You need to build a streaming pipeline from Pub/Sub → Dataflow → BigQuery. How would you design it?**

Step 1. Define requirements and SLAs - decide acceptable end-to-end latency (e.g., <10s), event ordering/idempotency needs, throughput, and failure/replay SLAs so design choices (micro-batch vs pure streaming, dedupe strategy) match business needs.

Step 2. Use Pub/Sub as the ingestion front door - producers publish events to a well-typed Pub/Sub topic; enable message ordering or use attributes with publish time and a unique event ID if ordering or de-duplication matters.

Step 3. Build a streaming Dataflow pipeline - read from Pub/Sub with a Dataflow (Apache Beam) streaming job that parses, validates, enriches (lookups, small in-memory caches), and tags events with processing metadata (ingest_time, source, event_id).

Step 4. Handle late and out-of-order events - in Dataflow use event-time windowing and set allowed lateness + side outputs for late elements; if strict event-time correctness is needed, buffer slightly or use watermark tuning rather than dropping late events.

Step 5. Implement deduplication and idempotency - include a stable event_id from producers and use insertId on BigQuery streaming inserts or perform MERGE into partitioned tables keyed by event_id so retries and at-least-once delivery do not create duplicates.

Step 6. Choose BigQuery write pattern - for lowest latency, use Dataflow's streaming writer to tabledata.insertAll (streaming inserts) into a partitioned table; for higher throughput and lower cost consider writing to a staging GCS parquet/avro and running frequent micro-batch load jobs or MERGE jobs.

Step 7. Partition and schema design in BigQuery - write into partitioned (by event_date) and clustered tables (by common filter/join keys) so downstream queries are efficient; keep schema nullable for evolving fields and store raw payload in a JSON column if needed.

Step 8. Monitor, alert and dead-letter - emit metrics (ingest rate, lag, errors) to Cloud Monitoring, route bad messages to a dead-letter Pub/Sub topic or GCS with error metadata, and create alerts for backpressure, high latencies, and DLQ size growth.

Step 9. Plan replay and backfill - persist raw event copies to GCS as a canonical source so you can reprocess or backfill into BigQuery if downstream schema or logic changes; ensure Dataflow jobs can be re-run idempotently.

Step 10. Test and scale - load-test expected peak traffic in staging, tune Dataflow worker autoscaling, batch sizes, and BigQuery slot/ingest limits; validate end-to-end latency, correctness, and cost trade-offs before production rollout.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 57. A legacy Spark job on Dataproc needs to write results into BigQuery. How would you do it?**

Step 1. Pick the right write method based on data size and latency - for large batch outputs use file-based export (write Parquet/Avro to GCS then load into BigQuery); for smaller or interactive jobs use the BigQuery connector to write directly.

Step 2. Use the native BigQuery connector for Spark for direct writes - configure the spark-bigquery-connector on Dataproc, set temporaryGcsBucket for intermediate staging, and use write with format("bigquery") to write to a partitioned/clusted destination table.

Step 3. For very large outputs prefer GCS staging + load jobs - have Spark write compressed, well-sized Parquet/Avro files to a GCS path, then trigger a BigQuery load job (batch) to ingest the files into a partitioned table; this is often faster and cheaper than streaming many rows.

Step 4. Handle schema and type mapping - ensure Spark schema maps correctly to BigQuery types (TIMESTAMP/DATE, STRING, RECORD); prefer Avro/Parquet which carry schema to reduce type-mismatch errors.

Step 5. Ensure idempotency and replaceability - write to a staging table or prefixed partition and use a MERGE or swap pattern (write to table_temp, then CREATE OR REPLACE/MERGE into production) to avoid partial writes and to make jobs restartable.

Step 6. Tune partitioning and sharding - output files organized by partition key (e.g., date) so BigQuery loads in parallel into partitions; avoid producing many tiny files—aim for 100MB–1GB per file for good parallelism.

Step 7. Secure credentials and permissions - run Dataproc with a service account that has BigQuery Data Editor and GCS write permission (for staging) scoped narrowly to required datasets and buckets.

Step 8. Monitor and retry - capture job status, row counts, and BigQuery load job outcomes; implement retries for transient errors and alert on schema mismatches or failed loads.

Step 9. Optimize cost and performance - prefer batch loads for very large results to reduce streaming costs, and use partition/cluster to lower downstream query costs on BigQuery.

Step 10. Document and automate - add the connector config, staging bucket, and load/merge steps into your CI/CD or orchestration so the Dataproc-to-BigQuery flow is reproducible and maintainable.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 58. You want to orchestrate a daily BigQuery ETL pipeline with dependencies. How would Composer help?**

Step 1. Use Cloud Composer (managed Apache Airflow) as the orchestrator - Composer schedules and runs DAGs (directed acyclic graphs) where each task represents an ETL step (export, transform, load, quality checks) and manages dependencies and retries.

Step 2. Model the pipeline as a DAG with clear task boundaries - create tasks for steps like extract from source, transform (Dataflow/Spark/SQL), load into BigQuery, run post-load validations, and notify; express dependencies so tasks run in the correct order.

Step 3. Use native BigQuery operators and sensors - Composer provides BigQuery operators (BigQueryInsertJobOperator, BigQueryCheckOperator) and GCS operators, so you can submit SQL, kick-off load jobs, and wait for completion without custom glue code.

Step 4. Implement retries, SLA and failure handling - leverage Airflow retries, on_failure callbacks, and SLA monitors to automatically retry transient failures and alert owners on persistent issues; build idempotent tasks to safely re-run.

Step 5. Manage secrets and connections securely - store service account keys, GCS buckets, and DB credentials in Composer connections or Secret Manager and reference them from DAGs securely.

Step 6. Add data quality and contract checks - run pre/post-check tasks (row count, null rate, schema check) and fail the DAG or route to remediation steps if checks fail, keeping downstream consumers safe.

Step 7. Parameterize and version DAGs - use variables and template fields for environment, date partitions, and resource names so the same DAG can run for different dates or environments; keep DAGs in source control for traceability.

Step 8. Coordinate dependent jobs and downstream consumers - use sensors to wait for external signals (file arrival in GCS, Pub/Sub message) and trigger downstream dashboards or notification tasks once data is ready.

Step 9. Monitor and visualize runs - Composer's Airflow UI shows DAG run history, task durations, and logs so ops can debug failures quickly; integrate with Cloud Monitoring for alerts and metrics.

Step 10. Scale and maintain - schedule heavy-transform tasks at off-peak times, tune worker size and Composer environment scaling, and periodically review DAGs for optimization and cost savings.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 59. Your real-time analytics pipeline needs both Dataflow and BigQuery. How do they complement each other?**

Step 1. Data ingestion and buffering - use Pub/Sub + Dataflow to receive and buffer events in real time, because Dataflow handles high-throughput ingestion, backpressure, retries, and windowing before anything reaches BigQuery.

Step 2. Stream processing, enrichment & dedupe - perform transformations, light joins (via side inputs), enrichment (cached lookups), schema validation, and deduplication in Dataflow so you send clean, near-final rows to BigQuery and avoid expensive rework there.

Step 3. Event-time semantics and late data handling - Dataflow provides event-time windows, watermarks, and allowed-lateness to correctly aggregate or route late/out-of-order events; BigQuery alone cannot easily implement complex windowing semantics in a streaming hot path.

Step 4. Write patterns to BigQuery - choose writes based on SLA: for sub-second visibility use streaming inserts (insertAll with insertId or MERGE for idempotency); for cost/performance balance use micro-batch files to GCS then load jobs. Dataflow can implement either.

Step 5. Analytical store and ad-hoc SQL in BigQuery - store the cleaned, partitioned & clustered data in BigQuery for fast ad-hoc SQL, large-scale joins, materialized views, and BI dashboards --BigQuery is optimized for analytical queries at scale.

Step 6. Offload heavy/periodic work to BigQuery - perform large joins, historical re-computations, aggregations, and ML training inside BigQuery where parallel compute and slot reservations give predictable performance and cost control.

Step 7. Low-latency serving & caching - combine BigQuery with BI Engine or materialized views to serve dashboards with sub-second latency for the most-used aggregates that were populated/maintained from the Dataflow stream.

Step 8. Observability, retries and replay - Dataflow provides fine-grained metrics, DLQs, and replayability (raw events in GCS/PubSub) so you can reprocess or backfill into BigQuery if logic or schema changes; BigQuery holds durable analytical history.

Step 9. Cost & design trade-offs - use Dataflow for compute-intensive, stateful streaming logic (cost = compute) and BigQuery for storage + scalable analytics (cost = storage + bytes scanned); choose streaming vs micro-batch depending on latency vs cost trade-offs.

Step 10. End-to-end operational patterns - implement schema evolution policies, idempotent writes (event IDs + MERGE), monitoring (Cloud Monitoring, BigQuery audit logs), and runbook automation so Dataflow and BigQuery work together reliably for real-time analytics.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 60. You want to schedule ML training inside BigQuery and push results to dashboards. How do you design this flow?**

Step 1. Prepare and stage features in BigQuery - build a clean, partitioned feature table (or feature view) in BigQuery by ETL (SQL or Dataflow) that contains labeled training data and any precomputed aggregations; keep raw payloads archived for reprocessing.

Step 2. Choose a BigQuery ML model and experiment interactively - use CREATE MODEL with the appropriate model type (e.g., LOGISTIC_REG / BOOSTED_TREE_REGRESSOR / ARIMA_PLUS) and run ML.EVALUATE, ML.TRAIN or AutoML if needed, iterating until metrics meet requirements.

Step 3. Validate and explain the model - run ML.EVALUATE, ML.FEATURE_IMPORTANCE, and ML.EXPLAIN to validate accuracy and get explainability artifacts for stakeholders before scheduling production runs.

Step 4. Automate training with Composer or Scheduled Queries - schedule model retraining (daily/hourly) via Cloud Composer DAGs or BigQuery scheduled queries; include steps to train, evaluate, and only promote models meeting threshold criteria.

Step 5. Version and store models and metadata - write model metadata (model name, version, training_time, metrics) to a model registry table in BigQuery and use a versioned naming convention (model_v20250912) so you can roll back or compare.

Step 6. Batch predict and persist results - after training, run ML.PREDICT to generate scored predictions into a partitioned results table (predictions table), include timestamps, model_version, and confidence scores so dashboards read stable outputs.

Step 7. Create fast-access summaries for dashboards - build materialized views or aggregated summary tables (daily/hourly slices) from the predictions table tuned to dashboard queries; these are small and fast to read.

Step 8. Enable low-latency serving with BI Engine - reserve BI Engine capacity and point BI dashboards (Looker Studio / Looker) to the materialized views or summary tables so dashboard tiles get sub-second response.

Step 9. Monitor model drift and data quality - add scheduled checks that compute performance metrics (accuracy, AUC, distribution stats) and alert when drift crosses thresholds, triggering retrain or rollback workflows in Composer.

Step 10. Governance, access, and retraining rules - enforce IAM for model and prediction tables, document model lineage, automate backfills from archived raw data for reproducibility, and have procedures to rollback to prior model versions if production metrics degrade.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 61. Your company wants to use BigQuery as both a data lake and warehouse. How do you design it?**

Step 1. Define zones and ownership - create clear zones (landing/raw, staging/curated, consumption/analytics) and map them to datasets or projects so ownership, access controls, and lifecycle rules are simple to apply.

Step 2. Ingest raw data with schema-on-read durable copies - land original files into GCS (Parquet/Avro/JSON) and optionally create external tables or raw BigQuery tables with a JSON or raw payload column so you keep a single immutable source of truth for reprocessing.

Step 3. Build a controlled ETL/ELT to curate data - transform raw into curated, typed, partitioned and clustered BigQuery tables (schema-on-write) via Dataflow/Batch SQL jobs; keep curated tables optimized for queries (denormalized where appropriate).

Step 4. Organize consumption layer for performance - expose marts: denormalized analytics tables, materialized views, and narrow data marts partitioned and clustered for dashboards and BI Engine; use authorized views for access control and stable APIs.

Step 5. Apply governance, metadata and data catalog - register datasets/tables and schemas in Data Catalog, enforce data contracts, tag sensitive fields, and apply IAM roles and row-level/column-level policies so lake + warehouse artifacts meet compliance.

Step 6. Manage lifecycle and cost - set partition and table expirations, archive older raw/curated partitions to chilled GCS (Parquet/Avro) and delete from BQ as needed; monitor storage/query cost and choose flat-rate vs on-demand based on query patterns.

Step 7. Support replay, reprocessing and schema evolution - retain raw source in GCS, version ETL code, store schema registry, and design idempotent pipelines (event IDs, MERGE) so you can reprocess or backfill without data loss.

Step 8. Provide observability and access patterns - instrument pipelines, surface metrics (ingest lag, slot usage, bytes scanned), publish example queries and best practices (partition filters, avoid SELECT *), and train teams so lake flexibility and warehouse performance are used appropriately.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 62. A data platform needs unified governance across GCS and BigQuery. How does BigLake help?**

Step 1. Explain what BigLake is and why it helps - -BigLake provides a unified data access layer that lets you manage and query data stored both in BigQuery and in GCS with a consistent catalog, metadata, and access controls so governance policies apply across your lake and warehouse.

Step 2. Centralize metadata and cataloging - -BigLake integrates with Data Catalog so you get a single place to register datasets, schemas, and business metadata for objects in BigQuery and files/tables on GCS, making discovery and classification consistent.

Step 3. Apply consistent IAM and fine-grained access controls - -BigLake enables you to impose the same IAM and policy tags (column-level and row-level controls) on external GCS tables as you do on native BigQuery tables so sensitive fields stay protected regardless of storage location.

Step 4. Enforce data policies and tags uniformly - -with Data Catalog policy tags and BigLake views you can mark columns as sensitive and ensure masking/authorization rules are enforced for both BigQuery-managed data and external GCS data.

Step 5. Simplify auditing and lineage - -because BigLake surfaces GCS objects as queryable tables, audit logs, lineage tracking, and access logs become easier to collect and analyze consistently across both storage systems for compliance reporting.

Step 6. Reduce duplication and keep single source of truth - -instead of copying raw files into BigQuery for governance, you can expose GCS data safely via BigLake external tables and apply the same governance, reducing duplication and drift.

Step 7. Support consistent schema evolution and validation - -BigLake + Data Catalog lets you manage schemas and schema changes centrally; you can validate schemas during ingestion and ensure consumer contracts are honored whether data lives in GCS or BigQuery.

Step 8. Operationalize lifecycle and retention policies - -combine dataset-level settings (partition expiration, table expiration) and GCS lifecycle rules with centralized governance so retention, archival, and access rules are implemented end-to-end.

Step 9. Provide predictable developer/analyst experience - -users query data the same way (SQL) whether it's in GCS or BigQuery, and governance controls (authorized views, policy tags) behave consistently, reducing accidental leaks or policy bypasses.

Step 10. Monitor, alert and iterate - -integrate BigQuery audit logs, Cloud Logging for GCS, and Data Catalog metadata into a governance dashboard to monitor access patterns, policy violations, and to continuously refine tag/policy definitions across the platform.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 63. Streaming data arrives late and out of order. How do you design pipelines to handle this?**

Step 1. Use event-time semantics and define allowed lateness - -design pipelines (Dataflow/Beam) to process by event time (not ingest time) and choose an allowed lateness window (for example 1 hour or 24 hours) that balances correctness and latency.

Step 2. Implement windowing and watermarks - -use fixed or sliding windows with watermarks so late and out-of-order events are incorporated into the correct time window up to the allowed lateness; watermarks drive when windows close for results emission.

Step 3. Route truly-late events to a side/output or DLQ - -for events outside the allowed lateness, send them to a dead-letter queue or quarantine topic/table so you can inspect and reprocess later instead of silently dropping or corrupting aggregates.

Step 4. Make operations idempotent and deduplicable - -include a stable event_id and use insertId or MERGE logic when writing to BigQuery so replays and retries don't create duplicates; this also helps when backfilling late data.

Step 5. Use stateful processing for incremental updates - -keep lightweight in-stream state (counts, sketches, HLL) in Dataflow so you can incrementally update aggregates when late events arrive without full recomputation.

Step 6. Support reprocessing/backfill from durable storage - -persist raw events to GCS (or a compacted Pub/Sub topic) as the canonical source so you can re-run batch jobs to correct aggregates or to fill windows missed due to lateness.

Step 7. Choose write strategy to BigQuery carefully - -for low-latency views use streaming inserts and incremental merges; for stronger consistency and easier backfill, micro-batch to GCS and run frequent load/MERGE jobs into partitioned BigQuery tables.

Step 8. Provide approximate/merge-friendly summaries - -for high-volume metrics use sketch-based summaries (HLL, t-digest) or incremental pre-aggregates that are easy to merge with late arrivals, trading a small accuracy cost for robustness and speed.

Step 9. Expose both near-real-time and finalized views - -serve a fast "working" view (near-real-time, may change with late arrivals) and a "finalized" view (after allowed-lateness/backfill window) so consumers know when numbers are stable.

Step 10. Monitor lateness metrics and automate remediation - -instrument end-to-end latency, watermark progress, DLQ size, and backfill success; alert when lateness grows and automate replay/backfill jobs and notifications to data producers so upstream issues are fixed.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 64. A client requires GDPR compliance with restricted access to PII. How do you implement this in BigQuery?**

Step 1. Classify and discover PII - -run a discovery (Data Catalog + DLP) to list which tables/columns contain PII and tag each field with sensitivity level so you know what to protect.

Step 2. Apply column-level controls with policy tags - -use Data Catalog policy tags to mark PII columns and enforce column-level access in BigQuery so only authorized roles can see sensitive columns.

Step 3. Use row-level access policies and authorized views - -create row-level policies for per-user/role filtering and expose data via authorized views that return masked or reduced data for users who must not see raw PII.

Step 4. De-identify where possible - -replace direct identifiers with pseudonyms or hashes, or use DLP to tokenize/ redact fields; keep mapping securely if reversible re-identification is required under controlled processes.

Step 5. Protect data-at-rest with CMEK and restrict export - -enable Customer-Managed Encryption Keys (CMEK) for datasets needing stronger key control and use IAM + VPC Service Controls to reduce risk of data exfiltration across projects/networks.

Step 6. Enforce least privilege via IAM roles - -grant the minimum BigQuery roles (and avoid dataset-level OWNER) and prefer service accounts for automated jobs; use separate projects/datasets for sensitive data to simplify controls.

Step 7. Audit and retain access logs - -enable Data Access and Admin audit logs, export them to a central audit BigQuery dataset and GCS; monitor read access to PII and set alerts for anomalous access patterns.

Step 8. Build data-subject and retention workflows - -implement automated retention (partition expiration, table expiration), deletion/backfill processes for DSARs (right to be forgotten), and documented procedures to locate & remove PII on request.

Step 9. Control downstream usage and exports - -use authorized views and query-time restrictions (maximumBytesBilled, query quotas), scan policies and require that exports (to GCS or Sheets) are only allowed after review or to approved buckets.

Step 10. Test, document and govern - -run periodic compliance checks, maintain a data inventory and access matrix, train teams on GDPR rules, and keep playbooks for audits, breach response, and data subject requests.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

**Question 65. Your team needs to share datasets across multiple projects while controlling cost. How would you approach it?**

Step 1. Centralize data in a shared project/dataset - -host canonical datasets in a central "platform" project so ownership, access control, and auditing are straightforward.

Step 2. Expose data via authorized views - -create views that expose only required columns, apply partition filters or row filters, and hide expensive joins so consumers can't inadvertently run full-table scans.

Step 3. Pre-aggregate and publish narrow data marts - -provide curated summary tables or materialized views for common queries so teams read small, optimized datasets instead of repeatedly scanning large raw tables.

Step 4. Use reservation/slot assignments or mixed pricing - -if you need predictable costs, buy flat-rate slots and assign reservations per team/project; alternatively keep on-demand but set budgets and alerts for spikes.

Step 5. Enforce query cost guards - -set default_maximum_bytes_billed or require maximumBytesBilled for user jobs, enable quotas, and use policies (Cloud IAM + org policies) to block runaway queries.

Step 6. Tag and track costs - -enforce labeling of queries/jobs and use the billing export + scheduled queries to allocate cost per team; surface reports so teams see their usage and are accountable.

Step 7. Apply dataset-level IAM and row/column policies - -restrict who can read/write and use policy tags/row-level access so only authorized teams can access high-cost or sensitive slices.

Step 8. Provide sandbox/provisioning patterns - -give teams separate child projects with controlled access to central datasets; use slot assignments, quota limits, and authorized views so they can develop without risking platform-wide cost spikes.

Step 9. Automate throttling/alerts and remediation - -create scheduled queries that detect high-cost queries (e.g., > threshold), send alerts, and optionally disable offending queries or notify owners for remediation.

Step 10. Educate and document best practices - -publish query templates, show how to add partition filters, promote use of materialized views, and provide a request workflow for extra resources (temporary slots, larger quotas) so teams follow cost-aware patterns.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar